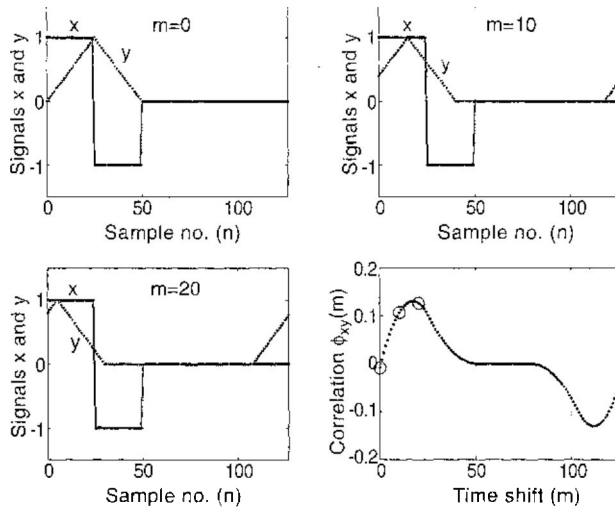


**FIGURE 3.1**

Signals $x(t)$ and $y(t + mT)$, with $m = 0, 10$, and 20 , and correlation function $\phi_{xy}(mT)$. Each circle on the correlation plot corresponds with one of the signal plots.

```
n=0:127;
% Waveform values with periodic extension.
x=[ones(1,25), -ones(1,25), zeros(1,78)];
y=[0:24, 25:-1:1, zeros(1,78)]/25;
yy=[y,y];
% Correlation function.
for m=0:127
    phi (m+1)=(1/128)*x*yy(m+1:m+128)';
end
```

(3.3)

Figure 3.1 illustrates the operation by showing the two signals aligned first with $m = 0$, next with $m = 10$, and finally with $m = 20$. In each picture, notice how x_n remains stationary, while y_{n+m} (with periodic extension) moves to the left. These three cases are circled on the plot of $\phi_{xy}(m)$ in the lower right corner of Figure 3.1. In each case, you can see how the value of $\phi_{xy}(m)$ is the sum of the product of the two curves on the corresponding plot.

Correlation has important uses of its own in signal processing. For example, a *correlation detector* is a system that correlates an incoming signal in real time with a stored "target," the objective being to detect the presence of the target in the incoming signal. An example of correlation detection is given in Figure 3.2. The target signal is shown in the upper plot. The target signal appears again in additive random noise at sample numbers 1000 and 3700 in the center plot. The signal can be seen in the center plot but is more easily

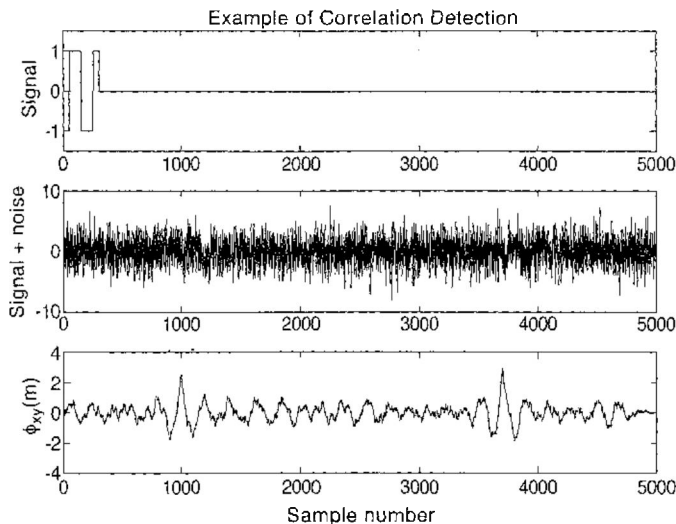


FIGURE 3.2

An example of correlation detection. The signal is seen more clearly in the correlation function (lower plot) than in the raw data (center plot).

detected in the lower plot, which is the correlation of the target signal with the center waveform.

3.3 The Discrete Fourier Transform (DFT)

The discrete Fourier transform (DFT) transforms sampled waveform or image data into spectral information about the respective waveform or image. We begin our discussion of the DFT with the complex Fourier series coefficients in Table 2.2:

$$c_m = c'_{-m} = \frac{1}{N} \sum_{n=0}^{N-1} x_n e^{-j2\pi mn/N}; \quad 0 \leq m \leq M-1 \quad (3.4)$$

Because $e^{-j2\pi mn/N} = \cos(2\pi mn/N) - j\sin(2\pi mn/N)$, and because the sum in (3.4) goes over n , we can see that the complex Fourier coefficient in (3.4) resembles the finite correlation function in (3.2). In fact, if we define $u_n = \cos(2\pi mn/N)$ and $v_n = \sin(2\pi mn/N)$, then, using the definition in (3.2), c_m is the sum of two correlation functions:

$$c_m = \phi_{xu}(0) + j\phi_{xv}(0) \quad (3.5)$$

Thus, recalling from (2.24) that the frequency corresponding with index m is

$$\omega_m = m\omega_0 = \frac{2\pi m}{NT} \quad (3.6)$$

we see that c_m is a measure of correlation of the vector $x = [x_0 \ x_1 \ \dots \ x_{N-1}]$ with unshifted sine and cosine vectors at the frequency ω_m . In this sense, c_m is a measure of the *spectral content* of x at ω_m rad/s.

The DFT is a scaled version of c_m , and is, therefore, also a measure of the spectral content of x . However, the DFT has exactly N components; that is, M in (3.4) is equal to N . The N components of the DFT are

$$X_m = \sum_{n=0}^{N-1} x_n e^{-j2\pi mn/N}; \quad m = 0, 1, \dots, N-1 \quad (3.7)$$

Thus,

$$X_m = Nc_m; \quad m = 0, 1, \dots, M-1 \quad (3.8)$$

We will discuss the relative values of M and N later in the section on the *sampling theorem*.

As we have done previously, we use X without a subscript to represent the DFT vector with components $X_0, X_1, \dots, X_{N-1}$. Thus,

$$X = [X_0 \ X_1 \ \dots \ X_{N-1}] \text{ is the DFT of } x = [x_0 \ x_1 \ \dots \ x_{N-1}] \quad (3.9)$$

3.4 Redundancy in the DFT

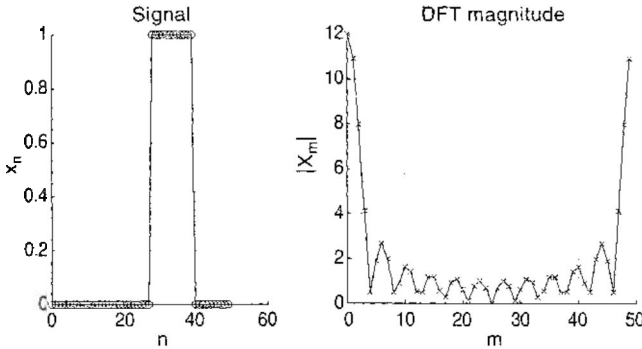
It is easy to show that the indices $m = 0, 1, \dots, N-1$ in (3.7) are sufficient to supply *all possible* values of the DFT, that is, that values of the DFT outside this range of m are repetitive. Let K be any integer. Then, because $e^{\pm j2\pi nK} = 1$,

$$X_{m \pm KN} = \sum_{n=0}^{N-1} x_n e^{-j2\pi(m \pm KN)n/N} = \sum_{n=0}^{N-1} x_n e^{-j2\pi mn/N} e^{\pm j2\pi nK} = X_m \quad (3.10)$$

Thus, the DFT is a periodic complex function of m with period N , and its values simply repeat outside the foregoing range of m .

Furthermore, when all the elements of the sample vector are real, essentially only half of the DFT components are independent. Again, because $e^{-j2\pi n} = 1$ for any value on n , we have

$$X_{N-m} = \sum_{n=0}^{N-1} x_n e^{-j2\pi(N-m)n/N} = \sum_{n=0}^{N-1} x_n e^{j2\pi mn/N} = X_m^* \quad (3.11)$$

**FIGURE 3.3**

Signal and DFT magnitude in Example 3.2.

where X'_m is the complex conjugate of X_m , as in Chapter 2, Table 2.1. Thus, when the samples (x_n) are real, all the independent values of X_m are in the following range:

$$m = 0, 1, \dots, \frac{N}{2} \quad (3.12)$$

To illustrate the DFT and the properties we have discussed so far, we use for our next example the sample vector x in the Fourier series example (2.31) of Chapter 2, which is shown on the left in Figure 2.5 and again on the left in Figure 3.3. The waveform vector, x , and the DFT vector, X , are computed as follows:

```
% Chapter 3 Example: Discrete Fourier Transform.
% Index vectors, n and m.
N=50;
n=[0:N-1];
m=[0:N-1];
% Sample vector, x.
x=[zeros(1,28),ones(1,12),zeros(1,N-40)];
% DFT, X.
X=x*exp(-j*2*pi*m'*n/N);
```

(3.13)

Notice again, as in Chapter 2, (2.31), how the index vectors (n and m) are multiplied to produce a square matrix, so that the “X” computation produces all N values of X_m in (3.7).

The DFT magnitude, $|X_m|$, is plotted on the right in Figure 3.3. As shown in (3.10), the DFT is periodic outside the range shown. Furthermore, the result in (3.11) is illustrated, because we are plotting the *magnitude* of the DFT, $|X_{N-m}| = |X'_m| = |X_m|$, and so the left and right halves of the plot are mirror images.

A final item to note in the plot of $|X_m|$ in Figure 3.3 is the value of X_0 , which is seen in (3.7) to be the sum of all the elements in the sample vector x . Because there are 12 ones in x and the rest of the elements are zeros, we have $X_0 = 12$.

3.5 The FFT Algorithm

The fast Fourier transform (FFT) is not a new transform. It is an algorithm for producing the DFT exactly as expressed in (3.7) but faster. The FFT appears in different forms in DSP hardware and software. In the literature,⁵⁻¹² the operation and structure of the FFT, as well as the interesting history of its development are described. Our purpose here is only to observe that the DFT computation, as expressed in (3.7), contains redundant complex products (products that must be essentially repeated in the computation of the DFT) and that repetitions of these products may be eliminated to produce a faster computation.

If (3.7) is used to compute the DFT in the form shown, we can see that there are N complex products in the sum and N sums in the transform, so there are N^2 products in all. Let us now express the DFT in the following form:

$$X_m = \sum_{n=0}^{N-1} x_n W_N^{nm}, \quad m = 0, 1, \dots, N-1, \quad \text{where } W_N = e^{-j2\pi/N} \quad (3.14)$$

We can see that W_N is a unit vector on the complex plane, and that W_N^k is periodic over k with period N ; that is, in MATLAB notation, $W_N^k = W_N^{\text{rem}(k, N)}$, where $\text{rem}(k, N)$ is the remainder after dividing k by N .

Redundant values of W_N^k are illustrated with $N = 8$ in Figure 3.4. With $N = 8$, the product $k = mn$ in the DFT (3.7) ranges from 0 through 49. However, only

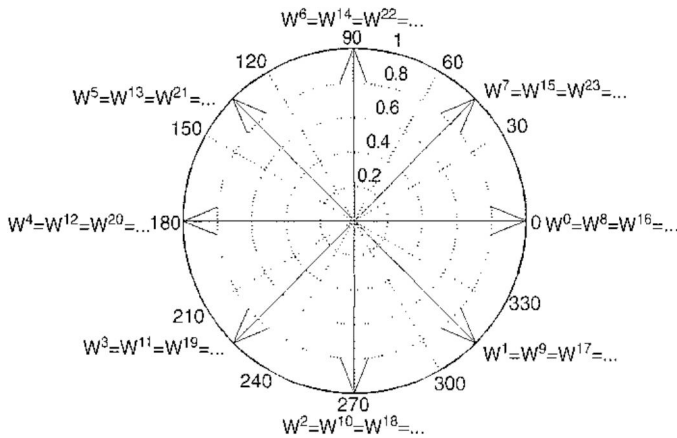


FIGURE 3.4

Repeating values of W_N^k with $N = 8$, showing that only N values are unique.

eight of these products are unique, as shown in the figure. Using this redundancy in W_N^k , the construction of a fast DFT algorithm begins by decomposing the sample vector. Suppose N is a multiple of 2. Then, we decompose the samples into two vectors containing even- and odd-numbered samples, and we write (3.14) as follows:

$$X_m = \sum_{n=0}^{N/2-1} x_{2n} W_N^{2mn} + W_N^m \sum_{n=0}^{N/2-1} x_{2n+1} W_N^{2mn}, \quad m = 0, 1, \dots, N-1 \quad (3.15)$$

In the definition of W_N^k in (3.14), we can see that $W_{N/2}^k = W_N^{2k}$ for any value of k . Thus, (3.15) becomes

$$X_m = \sum_{n=0}^{N/2-1} x_{2n} W_{N/2}^{mn} + W_N^m \sum_{n=0}^{N/2-1} x_{2n+1} W_{N/2}^{mn}, \quad m = 0, 1, \dots, N-1 \quad (3.16)$$

Although this result seems like a rather complicated way to express the DFT, it gives the basis for fast algorithms, because each sum in (3.16) is seen to be a DFT, but of size $N/2$ rather than N , requiring $(N/2)^2$ instead of N products. Each smaller DFT is periodic in m with period $N/2$, and so two periods are contained in the range $0 \leq m < N$. Also, there are $N/2$ unique outer products in the right-hand term, because, as illustrated in Figure 3.4, $W_N^k = -W_N^{k-N/2}$ when $k \geq N/2$. Thus, (3.16) expresses a total of $2(N/2)^2 + N/2 = N^2/2 + N/2$ complex products, which are fewer than the N^2 products in (3.14).

Suppose further that N is not only a multiple of 2 but also a *power* of 2, say $N = 2^K$. Then each of the sums in (3.16) can be decomposed as we have just described, and (3.16) can be written as the sum of four sums, and so on, until after K such steps, there are N sums with only one term in each sum. In this process, we have added $N/2$ complex products at each stage and finally produced N sums that contribute $N/2$ additional nonredundant products. Thus, if $N = 2^K$ and (3.14) is decomposed iteratively using (3.16) to produce the FFT, the DFT computation is reduced as follows:

$$\begin{aligned} \text{Complex products in DFT} &= N^2 \\ \text{Complex products in FFT} &= \frac{N}{2} K = \frac{N}{2} \log_2 N \end{aligned} \quad (3.17)$$

FFT algorithms are most efficient when N is a power of 2, which is assumed in (3.17), because then the number of decompositions ($\log_2 N$) is maximum. However, the process just described works when N has any prime factors other than itself. For example, suppose N is a multiple of 3. Then it is not hard to see that the equivalent version of (3.16) is as follows:

$$X_m = \sum_{n=0}^{N/3-1} x_{3n} W_{N/3}^{mn} + W_N^m \sum_{n=0}^{N/3-1} x_{3n+1} W_{N/3}^{mn} + W_N^{2m} \sum_{n=0}^{N/3-1} x_{3n+2} W_{N/3}^{mn}, \quad m = 0, 1, \dots, N-1 \quad (3.18)$$

Here, we see that there are $(N/3)^2$ products in each reduced DFT, plus N outer products in the second and third terms. Therefore, the original N^2 products are reduced to $3(N/3)^2 + 2N$ (that is, $N^2/3 + 2N$) products in this case.

FFT algorithms have different specific forms in software and hardware, but they are all based on decomposing the DFT iteratively into successively smaller DFTs as illustrated in (3.16) and (3.18). Robust FFT algorithms, such as the MATLAB *fft* function, take advantage of all prime factors of the signal vector size (N) to reduce the number of complex products.

In MATLAB there is a function called *flops* that allows one to keep track of the number of floating-point operations. The *flops* function may be used in the following MATLAB code to compute $f(N)$, the number of floating-point operations needed in the FFT computation of a vector x of length N . The first expression, *flops*(0), resets the counter, and the second gives $f(N)$.

```
flops(0);  
fft(x,N);  
f(N)=flops;
```

Figure 3.5 illustrates the results of computing $f(N)$ for a range of values of N . All values of $f(N)$ for $4 \leq N \leq 2500$ are plotted as circles. In addition, two boundary curves are plotted. The upper plot is of a constant times N^2 , and the lower is of a constant times $(N/2)\log_2 N$. The *flops* function counts more than just complex products, but in Figure 3.5, we can see, nevertheless,

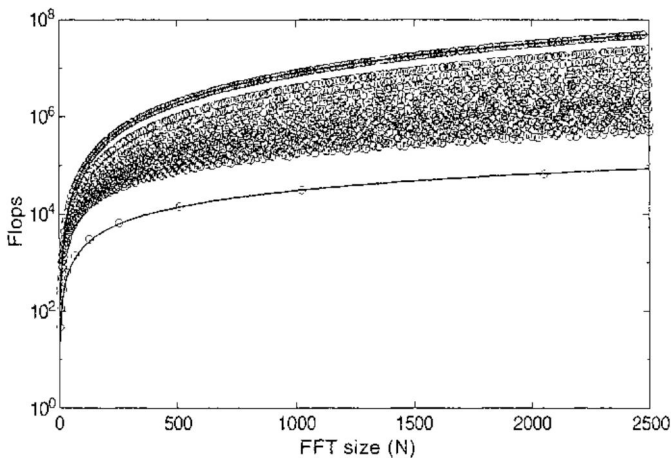


FIGURE 3.5

Floating-point operations (flops) as a function of FFT size (N). Lower boundary plot is proportional to $(N/2)\log_2 N$; upper boundary is proportional to N^2 .

that the values of $f(N)$ conform to the theory. Notice that when N is a power of 2, $f(N)$ lies close to the lower bound. On the other hand, when N is a prime number and has no factors other than itself, $f(N)$ is on the upper bound.

3.6 Amplitude and Phase Spectra

We turn now from the computation of the DFT to its use in spectral analysis. We introduce the subject in this chapter and continue it in Chapter 7. We begin with a discussion of the *amplitude spectrum* and the *phase spectrum* of a signal vector. These are easily defined. The amplitude spectrum is the vector with elements that are DFT component amplitudes, and the phase spectrum is the vector of DFT component phase angles in radian units. Thus, if $X = [X_0, X_1, \dots, X_{N-1}]$ is the DFT of a signal vector, $x = [x_0, x_1, \dots, x_{N-1}]$, then

$$\begin{aligned}\text{Amplitude spectrum} &= \text{abs}(X) = [|X_0| \ |X_1| \ \dots \ |X_{N-1}|] \\ \text{Phase spectrum} &= \text{angle}(X) = [\angle X_0 \ \angle X_1 \ \dots \ \angle X_{N-1}]\end{aligned}\quad (3.19)$$

When these DFT functions are applied to measured data, that is, samples of a continuous signal, each component is associated with a corresponding frequency. Frequency is usually measured in one of four systems of units. It is important to get these units straight in one's mind, because they are all used in the literature, and they cause a surprising amount of confusion among students and even engineers who should know them.

All of the four frequency scales are illustrated below the DFT index (m) in the example in Figure 3.6, which is a plot of the amplitude spectrum of

$$x = \left[\frac{\sin[2\pi(n - 50.5)/5]}{n - 50.5}; \quad n = 0, 1, \dots, 99 \right] \quad (3.20)$$

It is interesting to note that the spectral content of x is mostly below index $m = 20$; however, we leave this point for a later discussion and concentrate on the frequency scales below the figure.

First, note that the range of the DFT index (m) is $[0, N/2]$. As shown in (3.10) and (3.11), the amplitude spectrum simply repeats outside this range. That is, from $m = N/2$ to N , the amplitude spectrum is the mirror image of Figure 3.6, and outside the range $m = [0, N - 1]$, the DFT is periodic. Thus, in the range $m = [0, N/2]$, we have the complete amplitude spectrum.

The first two frequency scales below the index are dimensionless. These are based on one rotation of the unit vector $e^{j2\pi m/N}$ on the complex plane, as shown previously. The Hertz-second (Hz-s) scale is defined as m/N , and the radian (rad) scale is defined as $2\pi m/N$.

TABLE 3.1
Frequency Conversion Factors
Example: Ω (Frequency in rad) = $\pi T * f$ (Frequency in Hz)

From	To				
	Index (m)	Hz-s (v)	rad (Ω)	Hz (f)	rad/s (ω)
Index (m)	m	m/N	$2\pi m/N$	$m/(NT)$	$2\pi m/(NT)$
Hz-s (v)	Nv	v	$2\pi v$	v/T	$2\pi v/T$
rad (Ω)	$N\Omega/(2\pi)$	$\Omega/(2\pi)$	Ω	$\Omega/(2\pi T)$	Ω/T
Hz (f)	NfT	fT	$2\pi fT$	f	$2\pi f$
rad/s (ω)	$N\omega T/(2\pi)$	$\omega T/(2\pi)$	ωT	$\omega/(2\pi)$	ω

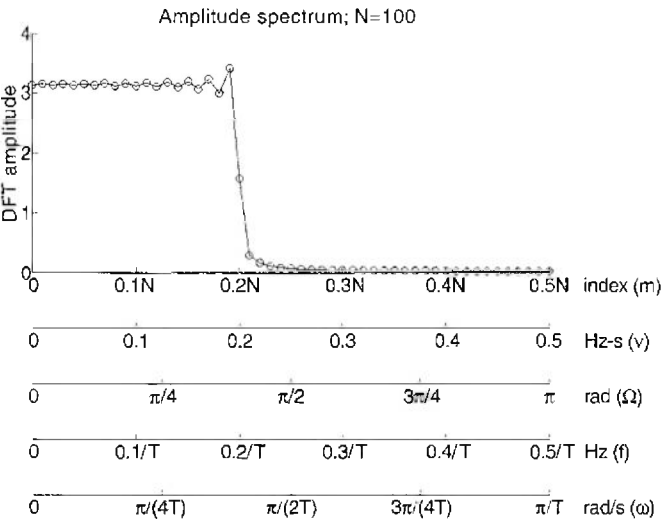


FIGURE 3.6
Amplitude spectrum of $X = \sin[2\pi(n - 50.5)/5]/(n - 50.5)$; $n = 0, 1, \dots, 99$, illustrating different frequency scales.

The last two frequency scales in Figure 3.6 are used with real data, where the time step (T) between samples is given in seconds (s). The Hertz (Hz) scale is defined as $m/(NT)$, and the radian-per-second (rad/s) scale is defined as $2\pi m/(NT)$.

The symbol used commonly to represent each frequency measure is shown at the right of each scale in Figure 3.6: v for Hz-s, Ω for rad, etc. All five scales are used in practice to describe frequency in whatever units seem convenient to the situation, so the DSP engineer must be able to move easily from one scale to another. Table 3.1 relates the scales by providing factors for all possible conversions. Note that each pair of symmetric table elements above and below the main diagonal represents a reverse conversion, and the factors are, therefore, inverses.

TABLE 3.2

Sampling Frequencies and DFT Frequency Increments

	Index (m)	Hz-s (ν)	rad (Ω)	Hz (f)	rad/s (ω)
Sampling frequency	N	1	2π	$1/T$	$2\pi/T$
Frequency increment	1	$1/N$	$2\pi/N$	$1/(NT)$	$2\pi/(NT)$

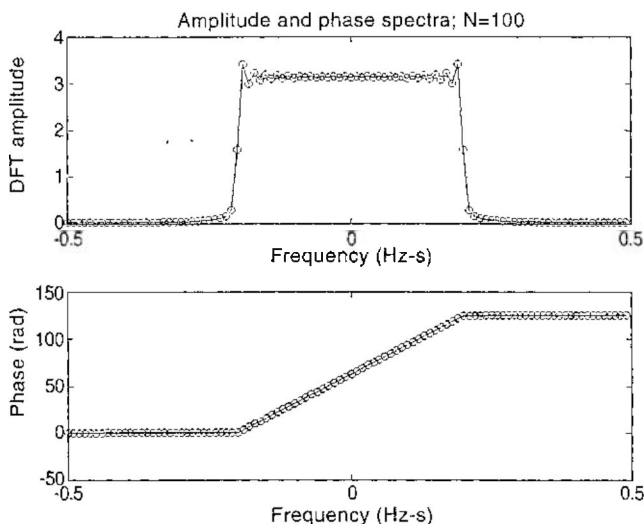


FIGURE 3.7

Amplitude and phase spectra of $X = \sin[2\pi(n - 50.5)/5]/(n - 50.5)$; $n = 0, 1, \dots, 99$.

Two other important features of the frequency scales—the *sampling frequency* and the *DFT frequency increment*—are given in Table 3.2 for the frequency index and for each frequency scale. The DFT index N designates the sampling frequency in the sense that, with $m = N$ in the DFT formula (3.7), the complex sinusoid $e^{j2\pi mn/N}$ goes through a cycle each time the sample number (n) is increased by one. Given $m = N$ at the sampling frequency, the rest of the sampling frequencies are found by using the top row of conversions in Table 3.1. The frequency increments in Table 3.2 are determined simply by dividing each sampling frequency by N , the number of frequency increments from zero to the sampling frequency. Because the index increment is one, these are identical with the conversion factors on the first row of Table 3.1.

Finally, Figure 3.7 is provided to illustrate the phase spectrum of the signal vector in (3.20) together with the amplitude spectrum. This time, the complete frequency range from -0.5 to 0.5 Hz-s (one period of the DFT) is shown

to illustrate the redundancy in (3.11), that is, $X_{N-m} = X_m^*$, which implies that the amplitude spectrum is evenly symmetric about the origin and the phase spectrum has odd symmetry about the origin. Amplitude and phase plots are often shown in this form.

The amplitude and phase spectra in Figure 3.7 may be computed with the MATLAB code shown below. The expressions are self-explanatory, with the exception of the *fftshift* and *unwrap* functions. The *fftshift* function swaps the upper and lower halves of the DFT vector (X). Thus (assuming N is even), the first $N/2$ DFT components cover $[0.5, 1)$ Hz-s, or, equivalently, $[-0.5, 0]$ Hz-s [see (3.10)], and the complete frequency range is therefore $[-0.5, 0.5)$, as shown.

The *unwrap* function is used to “unwrap” the output of the *angle* function. *Angle* computes the phase in radians of each DFT component using the “ATAN2” arctangent formula, which resolves all angles into the range $[-\pi, \pi]$ radians. The *unwrap* function adds multiples of 2π to eliminate “jumps” from $-\pi$ to π or vice versa in the progression of phase elements.

```
n=0:99;
x=sin(2*pi*(n-50.5)/5)./(n-50.5);
X=fftshift(fft(x));
amplitude=abs(X);
phase=unwrap(angle(X));
```

(3.21)

3.7 The Inverse DFT

The DFT in the form of (3.7) may be viewed as a mapping in N dimensions of a signal vector, x , into a DFT vector, X . Each vector has N components. Only half plus one of the DFT components are independent, as we have seen, but half minus one are complex and have two independent components each (X_0 and $X_{N/2}$ are the exceptions). Therefore, x and X have equal degrees of freedom, and an inverse mapping is possible.

The inverse mapping may be found by multiplying X_m in (3.7) by $e^{j2\pi mn/N}$ and summing over m . [Note that because n appears in the left-hand sum, we use k instead of n on the right when (3.7) is substituted for X_m .]

$$\sum_{m=0}^{N-1} X_m e^{j2\pi mn/N} = \sum_{m=0}^{N-1} \sum_{k=0}^{N-1} x_k e^{j2\pi m(n-k)/N} = \sum_{k=0}^{N-1} x_k \sum_{m=0}^{N-1} e^{j2\pi m(n-k)/N} = Nx_n \quad (3.22)$$

Once again, this result is an application of (1.21), where the geometric series on the right in (3.22) sums to zero for all values of k except $k = n$, where the

sum is N . So, we can now present the DFT as a symmetric pair:

$$\begin{aligned} X_m &= \sum_{n=0}^{N-1} x_n e^{-j2\pi mn/N}; \quad m = 0, 1, \dots, N-1 \\ x_n &= \frac{1}{N} \sum_{m=0}^{N-1} X_m e^{j2\pi mn/N}; \quad n = 0, 1, \dots, N-1 \end{aligned} \quad (3.23)$$

Thus, the inverse transformation from X back to x is the same as the forward transform, except for a change of sign in the exponent and the scaling factor $1/N$. The MATLAB *ifft* function implements the inverse transformation as shown, again using the fast algorithm described in Section 3.5, with the scaling factor included, so that, for example, the expression " $y = \text{ifft}(\text{fft}(x))$ " would (neglecting roundoff errors) be the same as " $y = x$."

3.8 Properties of the DFT

In addition to the inverse transform, the DFT has other properties worth mentioning. Along with properties described in Sections 3.4 and 3.6, four additional properties are summarized in Table 3.3. In this section, we discuss these properties. If the reader is already familiar with them, he or she may wish to skip the discussion.

The *periodicity* and *redundancy* properties of the DFT are given by (3.10) and (3.11). Note that if the signal vector x is complex, the redundancy property does not hold.

The *linearity* property follows from the DFT definition (3.7). If $ax_n + by_n$ is substituted for x_n in (3.7), we can write the sum in (3.7) as the sum of a times the DFT of x plus b times the DFT of y , thus proving the property. We can

TABLE 3.3

Properties of the DFT ($x = [x_0, x_1, \dots, x_{N-1}]$; $X = \text{DFT}(x) = [X_0, X_1, \dots, X_{N-1}]$)

1. Periodicity	$X_{m \pm KN} = X_m$
2. Redundancy	$X_{N-m} = X_m^*$, that is, $X_{-m} = X_m^*$ if x is real
3. Linearity	If $s = ax + by$, then $S = aX + bY$
4. Two real elements	$X_0 = x_0 + x_1 + \dots + x_{N-1}$ and $X_{N/2} = x_0 - x_1 + x_2 - \dots \pm x_{N-1}$ are real if x is real
5. Zero extension	If K zeros are appended to x , Δv decreases to $\Delta v = 1/(N + K)$ Hz-s
6. Zero insertion	If K zeros are inserted after each sample in x , the DFT repeats K times
7. Resampling	By inserting zeros properly into X , the time step in x can be changed
8. Linear phase shift	If x_{n-k} is substituted for x_n , $2\pi km/N$ rad are subtracted from $\angle(X_m)$

apply the linearity property to the DFT of a complex signal vector as follows. If $x = u + jv$, then the DFT of x must be the complex sum of two DFTs of real vectors, that is, $X = U + jV$. Because this is true, it is usually sufficient to use real signal vectors in our discussions of the DFT.

Property 4 states that when x is real, X_0 is the sum of the elements of x , and $X_{N/2}$ is the alternating sum of the elements as shown in the table. As discussed in Section 3.7, this property results in the DFT of a real signal vector having exactly N degrees of freedom in the inclusive index range $[0, N/2]$.

Properties 5 through 7 in Table 3.3 are interesting and useful characteristics of the DFT. Properties 5 and 7 are in a sense duals of each other. Each causes a form of *interpolation* in its corresponding transform domain.

First, we consider *zero extension in the time domain* (Property 5). Suppose we form a vector y by appending K zeros to a row vector, x . The MATLAB expression for doing this would be $y = [x, \text{zeros}(1, K)]$. [The *zeros* function in this case creates a row of K zeros as in the example (3.3), etc.] We now find the DFT of y using (3.7). Noting that the last K elements of y are zeros, the result is

$$Y_m = \sum_{n=0}^{N+K-1} y_n e^{-j2\pi mn/(N+K)} = \sum_{n=0}^{N-1} x_n e^{-j2\pi mn/(N+K)}; \quad m = 0, 1, \dots, N+K-1 \quad (3.24)$$

One way to interpret this result is to say that the DFT of y is the same as the DFT of the original vector x , but interpolated in frequency, that is, given at frequencies that are more closely spaced. For example, suppose $K = N$ so that the length of y is twice the length of x . Then, it is easy to see in (3.24) that $Y_{2m} = X_m$, that is, every other value in Y is an original DFT component, and the in-between values are “interpolated” DFT components.

An example of this with $K = 3N$ is given in Figure 3.8. The two signals and DFTs are produced with the following MATLAB expressions:

```
N1=32; N2=128;
x=[ones(1,N1/2), -ones(1,N1/2)];
X=fft(x,N1);
Y=fft(x,N2);
```

The original signal is shown at top left with $N_1 = 32$, and again with 96 zeros appended ($N_2 = 128$) at top right. The original amplitude spectrum is plotted at bottom left, and the interpolated version is plotted at bottom right. In the latter, note that the original points (circled) coincide with the left-hand spectrum, and that between each pair of original points, there are three interpolated points. Notice also the use of the “Hz-s” frequency scale to emphasize that frequency *range* does not change.

This effect of zero-extension in the time domain leads to a *continuous* version of the DFT, because there is theoretically no limit to the number of appended zeros and, therefore, no limit to the amount of interpolation. That is, if we

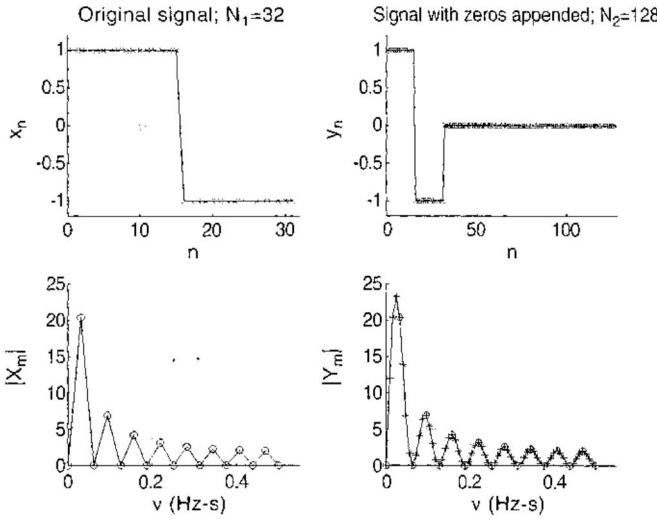


FIGURE 3.8

Zero-extension in time resulting in interpolation in frequency. With $N_2 = 4N_1$ there are four times as many points in the interpolated spectrum.

substitute ω (rad/s) into (3.7) in accordance with Table 3.1, we obtain a continuous form of the DFT:

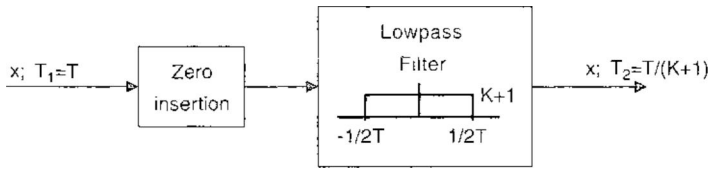
$$\text{Continuous DFT: } X_m = \sum_{n=0}^{N-1} x_n e^{-j\omega nT}; \quad 0 \leq \omega < 2\pi/T; \quad m = \frac{N\omega T}{2\pi} \quad (3.25)$$

The question of whether the continuous DFT gives a true representation of the spectrum of a sampled continuous signal, $x(t)$, must remain until we discuss the *sampling theorem* later in this chapter.

Zero insertion between samples of a signal vector in the time domain also has an interesting effect on the DFT. Suppose K zeros are inserted after each sample in x to produce y , so that $y_k = x_n$ when $k = (K+1)n$, and $y_k = 0$ when k is not a multiple of $K+1$. Then, the DFT of y is

$$Y_m = \sum_{k=0}^{(K+1)N-1} y_k e^{-j\frac{2\pi mk}{(K+1)N}} = \sum_{n=0}^{N-1} x_n e^{-j\frac{2\pi mn}{N}} = X_m; \quad m = 0, 1, \dots, (K+1)N-1 \quad (3.26)$$

We get the second sum by using $y_k = x_n$ when $k = (K+1)n$ and $y_k = 0$ otherwise. Thus, the DFT of x with zeros inserted is a succession of $K+1$ DFTs of x . Now suppose x is a sample vector with $T_1 = T$ seconds between samples. With zeros inserted in x , the interval is then reduced to $T_2 = T/(K+1)$, and the corresponding sampling frequencies are $f_1 = 1/T$ and $f_2 = (K+1)/T$ Hz. This, together with

**FIGURE 3.9**

Interpolation between samples of vector x by inserting K zeros between samples and then removing all spectral content above $1/(2T)$ Hz.

the result in (3.26), suggests the *time-domain interpolation* scheme illustrated in Figure 3.9, which will in turn take us to Property 7 in Table 3.3.

In Figure 3.9, an ideal lowpass filter is used following the insertion of K zeros after each sample in x . The filter has the effect of limiting the DFT of y , so it contains only the components of the first of the sequence of DFTs in (3.26), thus creating an *interpolated* version of x , that is, a vector that has $K + 1$ times as many samples of x and a proportionately smaller time step (T_2), and yet it has the same spectrum as the original x and the same values at the original sample points of x .

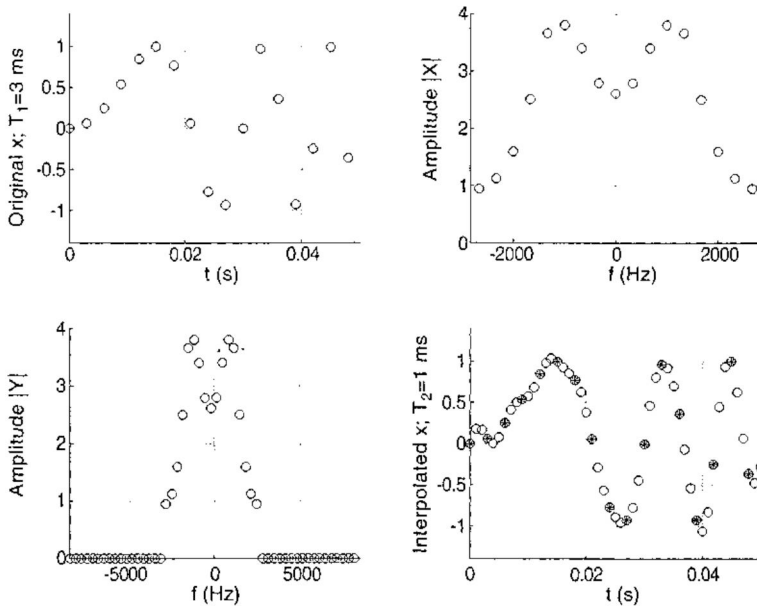
The entire process in Figure 3.9 can be accomplished in the frequency domain simply by expanding X , the DFT of x , with zeros, and effectively creating Y in (3.26) without any frequencies above $1/(2T)$ Hz, that is, without any nonzero components at indices above $m = N/2$. This we do by inserting zeros in the *middle* of X , as in the following MATLAB expressions:

```
X=fft(x);
Y=[X(1:(N+1)/2), zeros(1,K*N), X((N+1)/2+1:N)];
y=(K+1)*ifft(Y);
```

(3.27)

Here, we assumed that N (the number of elements in x) is odd, so $X(2:N)$ can be split into equal halves, and there is no component at $m = N/2$. (If N is even, $X_{N/2}$ must be split into two halves on either side of the zeros and the number of zeros reduced to $KN - 1$.) Notice that Y , the DFT of y , now has $(K + 1)N$ components and no frequencies above $1/(2T)$ Hz. Thus, when the inverse transform is taken, y contains K interpolated samples after each original sample in x , no frequencies above $1/(2T)$ Hz, and a smaller time step, $T_2 = T/(K + 1)$. The inverse transform is multiplied by $K + 1$, because the factor in front of the sum in (3.23) has changed from $1/N$ to $1/((K + 1)N)$.

An example of this interpolation process with $N = 17$ and $K = 2$ is shown in Figure 3.10. The original signal vector (x) is at the upper left with time step at $T_1 = 3$ ms. The amplitude spectrum, $|X|$, is shown on the upper right and on the lower left with zeros inserted. The spectra are shown using the *fftshift* function described previously to emphasize the frequency content, but the computations are done exactly as in (3.27). Finally, the interpolated vector (y) is plotted on the lower right, with x overplotted to show that the

**FIGURE 3.10**

Interpolation in the time domain by expanding the DFT with zeros in the frequency domain. $N = 17$, and $K = 2$ in this example. The process is equivalent to that of Figure 3.9.

samples of x and y coincide. Again, its properties are that it occupies essentially the same time frame as the original vector, the time step has been decreased to $T_2 = 1$ ms, and the spectral content has not been changed from the original.

At this point, it is natural to ask whether increasing K indefinitely in the process just described would allow one to recover the original continuous waveform from which the samples in x were taken. And the answer, as we shall see, is “yes,” provided the *sampling theorem* holds for the vector x .

The final DFT property, Property 8 in Table 3.3, is called *linear phase shift*. The simplest way to illustrate this property is to assume there is a *signal component* given by

$$x = [x_n; 0 \leq n < N]$$

within a long vector of length $L \gg N$, and also a time-shifted version of x given by

$$y = [y_n; 0 \leq n < N] = [x_{n-k}; k \leq n < N+k],$$

such that x and y are entirely within the long vector. We now examine the contribution of y to the DFT of the long vector using (3.7):

$$Y_m = \sum_{n=0}^{L-1} x_{n-k} e^{-j2\pi mn/L} = \sum_{i=0}^{L-1} x_i e^{-j2\pi mi/L} e^{-j2\pi mk/L} = X_m e^{-j2\pi mk/L}, \quad 0 \leq m < L \quad (3.28)$$

The second sum is found by substituting $i = n - k$ and keeping the same limits, because the indices of x and y are, by definition, within the limits $0 \leq i < L$. Thus, we have shown that delaying a signal component k samples causes no change in the component's amplitude spectrum and subtracts a linear phase component given by $2\pi mk/L$, where L is the size of the transform, from the phase spectrum.

Linear phase shift is illustrated in Figure 3.11 using transform size $L = 50$ and delay $k = 4$ samples. The signal component, x , is on the upper left, and its shifted version, y , is on the lower left. The respective phase spectra are on the right. In this example, the linear phase component, plotted at the lower right as a dashed line, is $-2\pi km/L = -0.50m$ rad.

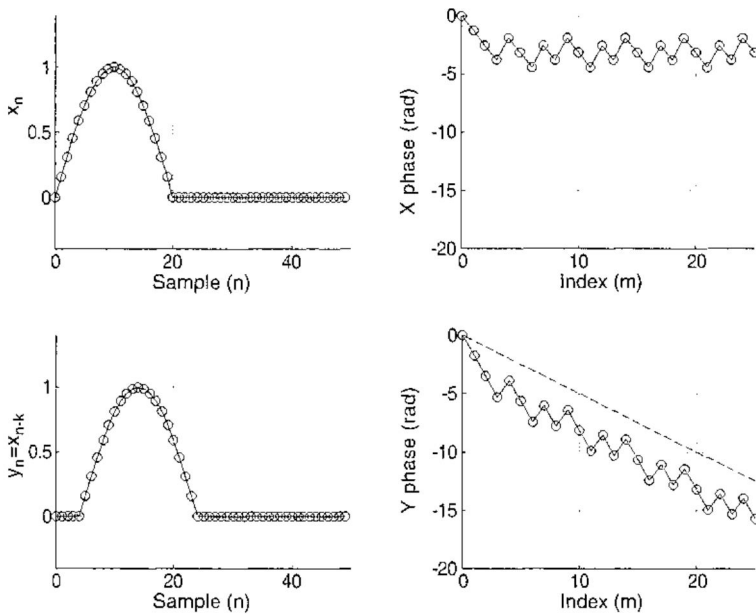


FIGURE 3.11

Illustration of linear phase shift. Signal samples $[x_n]$ are delayed to produce $[y_n]$ with $y_n = x_{n-k}$, $0 \leq n < N$. The delay does not alter the amplitude spectrum but subtracts $2\pi mk/N$ rad from the phase spectrum of x . The dashed line shows this linear phase component. $N = 50$, and $k = 4$ in this illustration.

3.9 Continuous Transforms

The focus of DSP is on vectors and arrays with elements taken from discrete sample spaces. But to keep in touch with reality, engineers must have ways to relate these vectors and arrays of discrete measured samples to their origin in the continuous real world, that is, to their corresponding continuous waveforms and images. This leads first to a discussion of the *Fourier Transform* (FT), which we introduce as a continuous, limiting version of the DFT. The Fourier transform of a continuous waveform, $x(t)$, is

$$\text{Fourier Transform: } \text{FT}\{x\} \equiv X(j\omega) = \int_{-\infty}^{\infty} x(t)e^{-j\omega t} dt \quad (3.29)$$

where ω is continuous frequency in rad/s, and t is time in s. [The use of X here for the FT raises the possibility of confusion with its use to represent the DFT vector, but this is conventional, so we must remember that the argument ($j\omega$) implies the FT rather than the DFT. Usually, the implication comes from the context in which X is used anyway.]

To relate the FT to the DFT, we define another continuous function of time, $\bar{x}(t)$, as a semi-infinite sequence of impulses equal to $x(t)$ at $t = 0, T, 2T, \dots$ given by

$$\bar{x}(t) = \sum_{n=0}^{\infty} x(t)\delta(t - nT); \quad 0 \leq t < \infty \quad (3.30)$$

where $\delta(t - nT)$ is the unit impulse at $t = nT$. Then the FT of $\bar{x}(t)$ is

$$\text{FT}\{\bar{x}\} = \int_0^{\infty} \sum_{n=0}^{\infty} x(t)\delta(t - nT)e^{-j\omega t} dt = \sum_{n=0}^{\infty} x(nT)e^{-j\omega nT} \quad (3.31)$$

The sum on the right results from exchanging the summation and integration operations and noting that in general $\int_{-\infty}^{\infty} f(t)\delta(t - \tau) dt$ is equal to $f(\tau)$ for any value of τ . Now, if we assume $x(nT) = x_n$ is a discrete sample of $x(t)$ and is nonzero only for $0 \leq n < N$, (3.31) is equivalent to

$$\text{FT}\{\bar{x}\} \equiv \sum_{n=0}^{N-1} x_n e^{-j\omega nT} \quad (3.32)$$

We recognize this as the DFT with continuous frequency, ω , that is, X_n in (3.25). Thus, we have the following interesting relationship between the FT and the DFT:

The Fourier transform (FT) of a sequence of impulse functions, each of which is an element of the sample vector x , is equal to the DFT of x at each frequency at which the DFT is measured. (3.33)

In addition, the FT has many other applications. The FT of a continuous signal provides the amplitude and phase spectra of the signal, just as the DFT provides the amplitude and phase in (3.19). That is, the amplitude spectrum is the FT magnitude, and the phase spectrum is the angle of the FT.

There is also an inverse FT, just as there is an inverse DFT. The inverse FT is

$$\text{Inverse FT: } x(t) = \frac{1}{2\pi} \int_{-\infty}^{\infty} X(j\omega) e^{j\omega t} d\omega \quad (3.34)$$

The inverse FT is verified when we substitute (3.29) for $X(j\omega)$ and switch the order of integration. Then, in the following expression, the term in square brackets is seen to equal one when $t = \tau$ and zero when $t \neq \tau$, that is, $\delta(t - \tau)$:

$$x(t) = \int_{-\infty}^{\infty} x(\tau) \left[\frac{1}{2\pi} \int_{-\infty}^{\infty} e^{j\omega(t-\tau)} d\omega \right] d\tau = \int_{-\infty}^{\infty} x(\tau) \delta(t - \tau) d\tau = x(t) \quad (3.35)$$

In some applications used later in this text, the use of the FT is limited, because even though the integral in (3.29) is infinite, the energy in the waveform must, in general, be finite in order for the integral to converge; that is, for convergence of the FT,

$$\int_{-\infty}^{\infty} |x^2(t)| dt < \infty \quad (3.36)$$

Thus, for example, the FT of any periodic function does not exist according to the definition in (3.29), although functions of ω that “work” as transforms of periodic functions in some sense are sometimes included in transform tables.

The *Laplace transform* is a modification of the FT that converges with any finite-valued waveform. The Laplace transform is the FT with frequency (ω) replaced by a complex variable, $s = \alpha + j\omega$. In this case, we can use $X(s)$ without ambiguity to represent the Laplace transform of $x(t)$, and thus define the Laplace transform as

$$\text{Laplace transform: } X(s) = \int_{-\infty}^{\infty} x(t) e^{-st} dt \quad (3.37)$$

Like the Fourier transform, the Laplace transform has an inverse similar to (3.34), but the integration is on the complex plane, being over s instead of $j\omega$. Extensive tables of the Laplace transform and its inverse are available in the literature.^{14,15}

The continuous transforms are useful mainly with continuous signals, and so they have limited use in a text like this when the main subject is DSP. However, the Fourier and Laplace transforms are useful in the discussion of the sampling theorem in the next section and also in the design of optimal

IIR filters, which we discuss later. The main purpose of this section has been to introduce the two transforms and to show how they are related to each other and to the DFT. The principal relationships are in (3.29), (3.33), (3.34), and (3.37).

3.10 The Sampling Theorem

Having discussed discrete and continuous transforms, which are used to map time-varying waveform and image data into the frequency domain, we are now in a position to discuss and apply the well-known *sampling theorem*. The sampling theorem can be stated in different ways, but its main use in DSP is to tell us the rate at which we must sample a continuous waveform in order to convey all the information in the waveform, that is, in order to be able to reconstruct the waveform from its samples.

Such a reconstruction is, of course, impossible, in general, because any set of samples “belongs” to an infinite set of continuous waveforms, as illustrated in Figure 3.12, which shows three different waveforms with the same sample values at $t = 0, 2, 4, \dots$. In other words, given a finite set of sample points, it is always possible to construct an infinite number of continuous functions through these points. If this is the case, then how can any continuous waveform ever be reconstructed uniquely from a set of sample points?

The answer lies in allowing only certain frequency components to exist in the continuous waveform. The sampling theorem may be stated simply

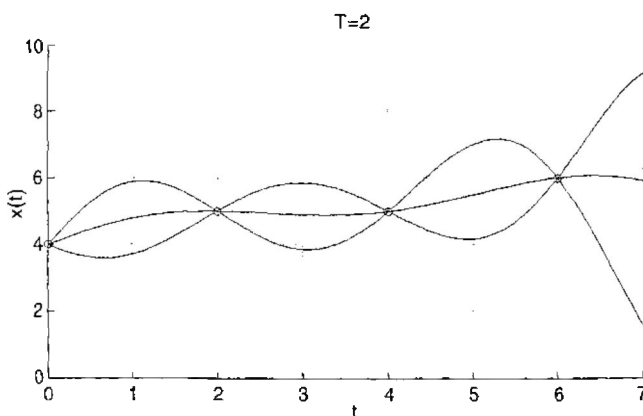


FIGURE 3.12

Different waveforms with the same samples at $t = 0, 2, 4, 6, \dots$

as follows:

If a continuous signal is sampled at a rate greater than twice its highest frequency component, then it is possible to recover the signal from its samples.

(3.38)

Notice that if the signal can be recovered from the samples, then all of information in the signal can be recovered from the samples. For example, if the signal is a speech signal conveying news about events, then all of the news is also conveyed in the samples, provided the sampling theorem is satisfied. Also, the samples in (3.38) are assumed to be accurate. In some cases, they are noticeably inaccurate, and *quantizing errors* must be taken into account. Quantizing errors are treated as statistical or random errors and are discussed in Chapter 10.

The sampling theorem is easy to prove, and in fact, might be anticipated from previous results in Sections 3.3 and 3.4. Suppose we have accurate samples of $x(t)$ taken at $t = 0, T, \dots, (N-1)T$, and we wish to recover $x(t)$ in the range $t = [0, NT)$. Let $\bar{x}(t)$ represent the corresponding set of impulse samples as in (3.30). Then, according to (3.33), the Fourier transform of $\bar{x}(t)$ and the DFT of the vector $x = [x_0 \ x_1 \ \dots \ x_{N-1}]$ are the same; that is, $\text{DFT}\{x\} = \text{FT}\{\bar{x}\} = \bar{X}(j\omega)$. Now suppose we express the impulse sequence in (3.30) as $\bar{x}(t) = x(t)d(t)$, where

$$d(t) = \sum_{n=-\infty}^{\infty} \delta(t - nT) \quad (3.39)$$

The Fourier series coefficients for this function, which is obviously periodic with period T and fundamental frequency $\omega_0 = 2\pi/T$, may be computed using (2.35):

$$c_n = \frac{1}{T} \int_{-T/2}^{T/2} \sum_{m=-\infty}^{\infty} \delta(t - nT) e^{-j2\pi m t/T} dt = \frac{1}{T} \sum_{m=-\infty}^{\infty} \int_{-T/2}^{T/2} \delta(t - nT) e^{-j2\pi m t/T} dt = \frac{1}{T} \quad (3.40)$$

In this result, the integrand on the right equals one at $t = 0$ and zero elsewhere. Using (2.34), the Fourier series for $\bar{x}(t) = x(t)d(t)$ is

$$\bar{x}(t) = \frac{1}{T} \sum_{m=-\infty}^{\infty} x(t) e^{j2\pi m t/T} \quad (3.41)$$

The final step is to take the Fourier transform of this version of $\bar{x}(t)$, as follows:

$$\begin{aligned} \bar{X}(j\omega) &= \frac{1}{T} \int_{-\infty}^{\infty} \sum_{m=-\infty}^{\infty} x(t) e^{j2\pi m t/T} e^{-j\omega t} dt \\ &= \frac{1}{T} \sum_{m=-\infty}^{\infty} \int_{-\infty}^{\infty} x(t) e^{-j(\omega - 2\pi m/T)t} dt = \frac{1}{T} \sum_{m=-\infty}^{\infty} X(j(\omega - 2\pi m/T)) \end{aligned} \quad (3.42)$$

Thus, $\bar{X}(j\omega)$, which is the Fourier transform of the sequence of impulse samples of $x(t)$, is $1/T$ times an infinite superposition of Fourier transforms of $x(t)$. But, if $X(j\omega)$, the Fourier transform of $x(t)$, is zero at and beyond half the sampling rate, that is, for $|\omega| \geq \pi/T$, then the terms in the sum in (3.42) do not overlap, and $X(j\omega) = T\bar{X}(j\omega)$ in this frequency range.

Thus, the sampling theorem is proved, because if we can compute $X(j\omega)$ from the DFT [that is, by transforming the sample sequence to produce $\bar{X}(j\omega)$], then we can recover $x(t)$ as the inverse transform of $X(j\omega)$. Two useful recovery methods that effectively accomplish these operations when the sampling theorem is satisfied are described in the next section.

3.11 Waveform Reconstruction and Aliasing

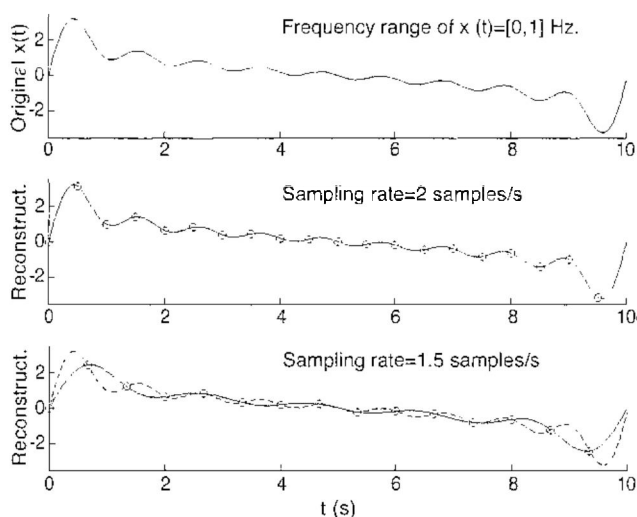
Our first waveform reconstruction method is based on the inverse DFT in (3.23) and on the equivalence of the DFT components with the Fourier series coefficients in (3.8). These suggest that writing the inverse DFT as a continuous function of t provides a continuous Fourier series for a function that passes through all the sample points and has no frequencies beyond half the sampling rate. Because (via the sampling theorem) the only function having these properties is $x(t)$, we conclude that the reconstruction, which we may call the *Fourier series reconstruction*, is exact when the sampling theorem holds. Furthermore, even when the sampling theorem does not hold, the Fourier series reconstruction is exact at all the sample points and is guaranteed to contain no frequencies at or above half the sampling rate.

Using the redundancy in (3.11), we write the inverse DFT (3.23) as follows, so that the frequency index stays at or below half the sampling rate index, that is, $\leq N/2$:

$$\begin{aligned} x_n &= \frac{1}{N} \sum_{m=0}^{N-1} X_m e^{j2\pi mn/N} \\ &= \frac{1}{N} \left(X_0 + 2 \sum_{m=1}^{m \leq N/2} \operatorname{Re}\{X_m\} \cos\left(\frac{2\pi mn}{N}\right) - \operatorname{Im}\{X_m\} \sin\left(\frac{2\pi mn}{N}\right) \right) \quad (3.43) \end{aligned}$$

The second line is obtained from the first by separating the first sum into two halves and applying (3.11) with (3.10). If N is even, $X_{N/2}$ must be zero for the sampling theorem to hold. Next, we substitute $\omega_0 = 2\pi/NT$ as in (2.24), and we let $t = nT$ to obtain from (3.43) the Fourier series reconstruction at any point in time:

$$x(t) = \frac{1}{N} \left(X_0 + 2 \sum_{m=1}^{m \leq N/2} \operatorname{Re}\{X_m\} \cos m\omega_0 t - \operatorname{Im}\{X_m\} \sin m\omega_0 t \right) \quad (3.44)$$

**FIGURE 3.13**

Two examples of Fourier series reconstruction using (3.44). In the center plot, the sampling theorem is satisfied, and reconstruction is exact. In the lower plot, the sampling rate is too low, and the reconstruction is not exact.

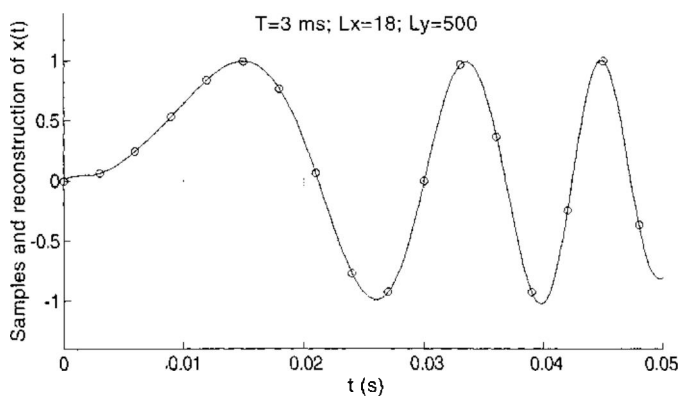
The following MATLAB function implements (3.44):

$$y = \text{reconst}(x, t) \quad (3.45)$$

In this function, t is any vector of time points, and y is the reconstructed waveform, $x(t)$, computed as in (3.44).

In Figure 3.13, there are two examples of the use of (3.44). The original waveform, $x(t)$, is shown in the upper plot. The spectrum of $x(t)$ is zero at and above 1.0 Hz. In the center plot, $x(t)$ is sampled at 2 samples/s, and $x(t)$ is reconstructed using (3.44). Because the sampling theorem is satisfied in this case, the reconstruction is exact. In the lower plot, $x(t)$ is sampled at 1.5 samples/s, and $x(t)$ is again reconstructed using (3.44). Because the sampling theorem is not satisfied in this case, the reconstruction (solid curve) differs from $x(t)$ (dashed curve), both of which are shown. However, the reconstruction is seen to pass through the sample points and to interpolate smoothly between the sample points. One would expect the latter, because, in this case, the reconstruction (3.44) contains only frequencies below 0.75 Hz.

The second reconstruction method was developed previously in Section 3.8. It is based on the *resampling* method on line 7 of Table 3.3, in which zeros are inserted into the DFT, resulting in interpolation between samples in the time domain. Because this method produces a more densely packed sample

**FIGURE 3.14**

Samples and reconstruction of a continuous signal, $x(t)$, using the “resampling” method of interpolation between samples. The reconstructed signal has no spectral content above half the sampling rate (166 Hz in this example).

sequence that includes the original sample points and has an unchanged spectrum, we can now use the sampling theorem to say that, if the sampling theorem is satisfied and if an increasingly large number of zeros is inserted into the DFT in the manner described [see (3.27), for example], the resampled function will, in the limit, become the original continuous function.

The resampling method is especially easy to implement using MATLAB expressions. The *resamp* function in the software that comes with this text is one implementation. The expressions in *resamp* are essentially similar to the expressions in (3.27), except even as well as odd sample vector lengths are allowed. An example of the use of *resamp* is provided in Figure 3.14. The implementing MATLAB expression is

$$y = \text{resamp}(x, Ly); \quad (3.46)$$

in which x is the sample vector (with length $Lx = 18$ in Figure 3.14), y is the interpolated version of x , and Ly is the specified length of y , which was set at 500 for this example. With 500 points, the reconstruction appears continuous.

Both reconstruction methods—Fourier series and resampling—produce *band-limited* approximations of the original signal from which the samples were taken. That is, the spectra of the reconstructions are limited to frequencies below half the sampling rate. These methods differ from other interpolation methods such as linear interpolation, splines, etc., which are not band limited.

A method used often in the design of DSP systems is shown in Figure 3.15. To prevent errors in the reconstruction (as well as other kinds of processing such as filtering), a signal is “prefiltered” to remove all frequency components over half the sampling rate. Then, as suggested in the illustration, the prefiltered signal can be sampled, stored, and later reconstructed in its original form.

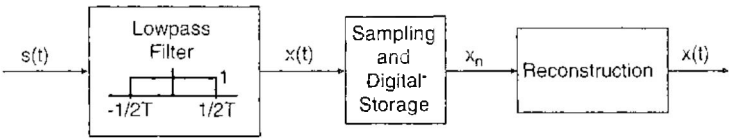


FIGURE 3.15
Reconstruction of a signal, $x(t)$, that has been prefiltered to remove all spectral content over half the sampling rate.

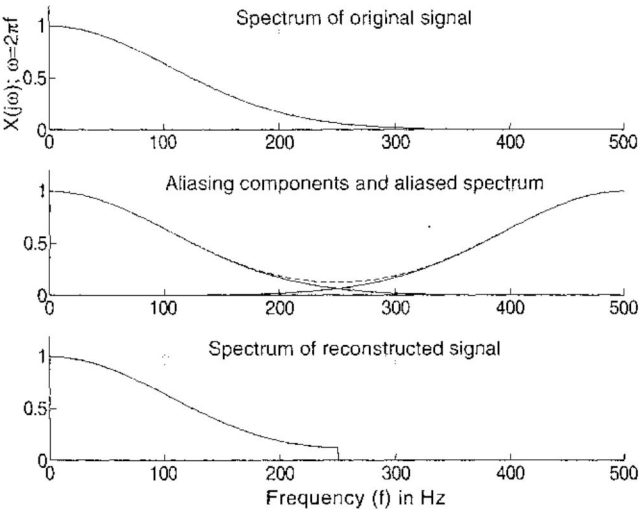


FIGURE 3.16
Illustration of aliasing in accordance with (3.40); sampling rate = 500 Hz. Upper: Spectrum of original signal. Center: Components of sum in (3.40) and sum (dashed curve). Lower: Spectrum of reconstructed signal.

The concept in Figure 3.15, although often used, cannot be done ideally as shown. Ideal filters do not exist, as we will see later in this text. So the question arises: How is a signal distorted, or *aliased*, when it is reconstructed after having been sampled at a rate that is somewhat less than twice its highest spectral content? The answer lies essentially in (3.33) and (3.42). Each of the reconstruction methods produces a signal with a spectrum that is the continuous DFT of the samples. But, from (3.33) and (3.42), the spectrum of the samples is identical with $1/T$ times the superposition of shifted Fourier transforms of the original signal in the range $|\omega| < \pi/T$, and is zero outside this range.

This phenomenon, called *aliasing*, is illustrated in Figure 3.16. The upper plot shows the spectrum of the original signal, $x(t)$. The center plot shows the two essential components in the sum in (3.42) (with $m = 0$ and $m = 1$)

and the sum, which is $\bar{X}(j\omega)$ in (3.42), assuming that $x(t)$ is sampled at 500 samples/s. The lower plot shows the reconstructed spectrum, which is distorted above about 150 Hz and is zero above 250 Hz, or half the sampling rate.

3.12 Exercises

General Instructions: (1) Whenever you make a plot in any of the exercises, be sure to label both axes so we can tell exactly what the units mean. (2) Make continuous plots unless otherwise specified. "Discrete plot" means to plot discrete symbols, unconnected unless otherwise specified. (3) In plots, and especially in subplots, use the *axis* function for best use of the plotting area.

1. Write a program similar to (3.13), and use the result to produce a figure similar to Figure 3.3, illustrating the signal and the magnitude of its DFT. In place of the signal in (3.13), use

$$x_n = 2 \sin\left(\frac{2\pi n}{10}\right); \quad n = 0, 1, \dots, 49$$

2. Do Exercise 1, this time using the following signal with $N = 500$:

$$x_n = 2 \sin\left(\frac{2\pi n}{50}\right) e^{-n/100}; \quad n = 0, 1, \dots, 499$$

3. Create a 2×2 array of plots using *subplot*(2,2,*). On the upper left, plot the signal vector x , where

$$x_n = \sin\left(\frac{2\pi n}{20}\right); \quad n = 0, 1, \dots, 99$$

versus time, assuming time step $T = 0.05$ s. On the lower left, make a discrete plot of the DFT magnitude, $|X_m|$, versus m for $m = 0, 1, \dots, 50$. On the upper right, make a discrete plot of the DFT magnitude versus frequency in the range $[0, 0.5]$ Hz-s. On the lower right, make a discrete plot of the DFT magnitude versus frequency in Hz from zero to half the sampling rate.

4. In this exercise, we examine the phenomenon known as *leakage*. The term refers to the spectrum as represented by the DFT, in which the "true" spectrum of a periodic signal or component is spread out as a consequence of transforming a signal vector of finite length and not containing an integral number of cycles of the periodic component.

Create a 2×2 array of subplots. On the upper left, plot the signal vector with $N = 92$:

$$x_n = \sin\left(\frac{2\pi n}{20}\right); \quad n = 0, 1, \dots, 91$$

Below this, make a discrete plot of the amplitude spectrum, $|X_m|$, versus frequency in the range $[0, 0.5]$ Hz-s, and note the leakage around the signal frequency, 0.05 Hz-s. Repeat these two plots on the right with the signal vector length increased to $N = 292$, and note the effect on leakage of the increased vector length.

5. Make two subplots, side by side. On the left, plot the following sample vector:

$$x_n = \sin\left(\frac{2\pi n}{25}\right)e^{-n/75}; \quad n = 0, 1, \dots, 299$$

On the right, make a connected discrete plot of the amplitude spectrum in terms of the frequency in rad. Use the correct symbol in Table 3.1 to label the frequency axis.

6. Do Exercise 5, assuming an interval of $0.1 \mu\text{s}$ between samples, and label the time and frequency axes appropriately. Use frequency units f (MHZ).
7. Complete the following:
 - a. Do Exercise 5, assuming an interval of $0.1 \mu\text{s}$ between samples, but instead of the amplitude spectrum, plot the unwrapped phase spectrum in terms of degrees versus frequency in MHZ.
 - b. Prove that for any real vector x , the DFT at $\omega T = 0$ and $\omega T = \pi$ must be real, and therefore, the phase must be either zero or $\pm 180^\circ$.
8. Beginning at $t = 0$, 2048 samples of $f(t) = te^{-t/3}$ are collected with time step $T = 0.015$ s. In the upper of two subplots, make a connected discrete plot of the amplitude spectrum of $f(t)$ over the range $[0, 0.6]$ Hz. In the lower subplot, make a connected discrete plot over the same frequency range of the interpolated amplitude spectrum using Property 5 in Table 3.3, such that the density of points is four times the density in the upper plot.
9. A function given by

$$x(t) = e^{-t/4} \sin(2\pi t/5); \quad t \geq 0$$

is sampled once per second from $t = 0$ through $t = 16$ s. In the upper of two subplots, make a discrete plot of x , the sample vector. In the lower subplot, make a discrete plot of the resampled version

- of x , with three samples between each pair of original samples. Use the resampling method (Table 3.3, Property 7) illustrated in (3.27).
10. Starting with the definition of the Fourier transform in (3.29), derive the Fourier transform of the continuous signal, $x(t)$, in Exercise 9. (You may wish to use the example of integration by parts in Chapter 1.) Plot the amplitude spectrum, $|X(j\omega)|$ versus ω in the range $[0, 10]$ rad/s. Is the sampling interval in Exercise 9 adequate to prevent aliasing? Why or why not? How would you select an “adequate” sampling rate for $x(t)$?
 11. The spectrum of $x(t)$ is given by $X(j\omega) = X(j2\pi f) = 10e^{-1/f^{1/2}}$, where f is in Hz. Note that the spectrum is real in this example. In each of two subplots, plot the continuous DFT of $x(t)$, that is, $\bar{X}(j\omega)$ in (3.42), over the frequency range $[0, 40]$ Hz. Use sampling intervals $T = 0.04$ s and 0.07 s in the upper and lower subplots, respectively. Use as many terms in the sum in (3.42) as necessary to complete the plot.
 12. A continuous waveform is described for $t \geq 0$ s as follows:

$$x(t) = 100te^{-150|t-0.02|} \sin(200\pi t)$$

- Create a sample vector, x , of $x(t)$ in the range $t = [0, 0.07]$ s using time step $T = 0.1$ ms. In the upper of two subplots, plot $x(t)$ as a continuous curve through the points in x . In the lower subplot, make a connected discrete plot of the amplitude spectrum in terms of the complete DFT of x versus frequency in the range $[0, 10]$ KHz. Based on the lower plot, estimate the lowest sampling frequency for $x(t)$ that would not violate the sampling theorem.
13. Compare the DFTs of two sample vectors taken from $x(t)$ in Exercise 12, both over the same range $t = [0, 0.07]$ s, but this time, with $T = 3.0$ and $T = 2.0$ ms. Plot the DFT amplitudes in two subplots, one above the other, both versus frequency in KHz over the range from zero to the sampling frequency. Observe the values of the DFT amplitudes around half the sampling rate and comment on whether either or both of the two time steps are small enough to satisfy the conditions of the sampling theorem.
 14. This is a reconstruction exercise using the resampling method (Table 3.3, Property 7). In the center of three subplots, plot the sample vector of the waveform in Exercise 12 with $T = 0.1$ ms. For the upper subplot, create a sample vector over the same 70-ms time interval using $T = 3.0$ ms, and plot the reconstructed continuous $x(t)$ using Property 7. Finally, do the same in the lower plot with $T = 2.0$ ms. Make all plots with time range $[0, 70]$ ms and amplitude range $[-2, 2]$. Comment on the adequacy of the sampling frequencies for subplots 1 and 3 by comparing both with subplot 2.

15. Do Exercise 14 using the Fourier series reconstruction instead of the resampling method. Comment on which reconstruction method is preferable for this particular signal, and why the method is preferable.

References

1. Lyons, R.G., *Understanding Digital Signal Processing*, Addison-Wesley, Reading, MA, 1997, chaps. 3 and 4.
2. Cartinhour, J., *Digital Signal Processing: An Overview of Basic Principles*, Prentice Hall, Upper Saddle River, NJ, 2000, chap. 8.
3. Oppenheim, A.V. and Schaffer, R.W., *Discrete-Time Signal Processing*, Prentice Hall, Englewood Cliffs, NJ, 1989, chap. 8.
4. Orfanidis, S.J., *Introduction to Digital Signal Processing*, Prentice Hall, Upper Saddle River, NJ, 1996, chap. 9.
5. Proakis, J.G., Rader, C.M., Ling, F., Nikias, C.L., Moonen, M., and Proudler, I.K., *Algorithms for Statistical Signal Processing*, Prentice Hall, Upper Saddle River, NJ, 2002, chap. 2.
6. Brigham, E.O., *The Fast Fourier Transform*, Prentice Hall, Englewood Cliffs, NJ, 1974, chap. 10.
7. Ahmed, N. and Rao, K.R., *Orthogonal Transforms for Digital Signal Processing*, Springer-Verlag, New York, 1975.
8. Elliott, D.F. and Rao, K.R., *Fast Transforms*, Academic Press, New York, 1983.
9. Bergland, G.D., A guided tour of the fast Fourier transform, *IEEE Spectrum*, 6, 41, July 1969.
10. *IEEE Transactions on Audio and Electroacoustics* (Special Issues on the Fast Fourier Transform), AU-15, June 1967 and AU-17, June 1969.
11. Cooley, J.W. and Tukey, J.W., An algorithm for the machine calculation of complex Fourier series, *Math. Comput.*, 19, 297, Apr. 1965.
12. Good, I.J., The interaction algorithm and practical Fourier series, *J. Roy. Statist. Soc.*, ser. B., 20, 361, 1958 and 22, 372, 1960.
13. Whittaker, E.T., Expansions of the interpolation-theory, *Proc. Roy. Soc. Edinburgh*, 35, 181, 1915.
14. Nixon, F.E., *Handbook of Laplace Transformation*, Prentice Hall, Englewood Cliffs, NJ, 1965.
15. Holbrook, J.G., *Laplace Transforms for Electronic Engineers*, Pergamon Press, Oxford, 1966.

Linear Systems and Transfer Functions

4.1 Continuous and Discrete Linear Systems

Digital signal processing technology has its roots in the analysis of continuous linear systems. We saw in the previous chapter how continuous signals may be regarded as limiting forms of corresponding discrete sample vectors, and the same idea holds for continuous and discrete processing systems. Continuous linear systems¹⁰⁻¹¹ are described with *linear differential equations*, and discrete linear systems are described with *discrete linear equations* that are similar to *linear difference equations*.

There are many reasons to keep these “continuous roots” in our thoughts as we proceed to develop the subject of DSP. In this chapter particularly, the time and frequency domains are similar for digital and continuous systems. Convolution in one domain (time, for example) is equivalent to multiplication in the other domain (frequency) in the case of either discrete or continuous signals. Most of the other notions in continuous linear systems, such as resonance, stability, impulse response, step response, transfer function, filtering, etc., are carried over and applied in the analysis of discrete systems.

Our discussion of linear systems begins with a description of the discrete linear equation and goes on to develop the linear transfer function as well as some of the rest of the concepts just mentioned. The development will be complete and (we hope) understandable in the terms given; nevertheless, it will be helpful, as it has been helpful in previous chapters, always to think of the sample vectors being derived from continuous signals.

4.2 Properties of Discrete Linear Systems

A *linear* equation describing the behavior of a discrete linear system is an equation that describes a straight line in a Cartesian coordinate system when the *dependent variable* in the equation is plotted versus the *independent variable(s)*. Take for example the following equation:

$$y_k = bx_k + ay_{k-1} \quad (4.1)$$

In this example, discrete values of the dependent variable, y , are related to discrete values of the independent variable, x . In the terminology of signal processing, when (4.1) describes a *linear system*, x is called the *input signal*, and y is called the *output signal*.

The two constants in (4.1), b and a , are called *coefficients*, or *weights*. We can tell the equation is linear, because the values of x and y appear only as first powers. But in what sense does (4.1) represent a straight line? To answer, we must "solve" the equation so that y is given explicitly in terms of x .

Before we solve (4.1), we should note that the subscript k denotes the position in an ordered sequence (vector) as in previous chapters, and this, in turn, implies an *order of processing* in equations like (4.1). Beginning at $k = 0$ (for convenience), y_0 is computed (usually with y_{-1} assumed to be zero) as $y_0 = bx_0$, then, $y_1 = bx_1 + ay_0$, $y_2 = bx_2 + ay_1$, and so on. We also associate continuous time with the order of processing, that is, $t = kT$ as in previous chapters, T being the time step between any two adjacent samples.

Several other properties of linear equations are related to the association of k with the time domain. The first is *realizability*. The linear operation in (4.1) is *realizable*, because the current value, y_k , of the output signal is computed in terms of its own past values (y_{k-1} , in this case), and not in terms of any future values of y , such as y_{k+1} , y_{k+2} , etc.

The second property is *causality*. The linear operation in (4.1) is *causal*, because y_k is computed in terms of present and past input signal values (x_k , in this case), and not in terms of any future values of x . The term comes from the idea that in nature, results in the present are not affected by events that have not yet happened. For example, if x_k in (4.1) were changed to x_{k+1} , then (4.1) would be a *noncausal* operation. Examples of causality and realizability are given at the end of this section.

The linear equation (4.1) is not hard to solve by induction, that is, by substituting repeatedly for the sample of y on the right. The result (assuming $x_k = 0$ for $k < 0$) becomes

$$\begin{aligned} y_k &= bx_k + a(bx_{k-1} + ay_{k-2}) \\ &= bx_k + a(bx_{k-1} + a(bx_{k-2} + a(bx_{k-3} + \dots))\dots) \\ &= b \sum_{n=0}^k a^n x_{k-n} \end{aligned} \quad (4.2)$$

Now, we see how (4.1) can be viewed as the equation of a straight line. The Cartesian signal space has $k + 2$ dimensions, and y_k is a function of $x_0, x_1, \dots, x_k$ in this space.

A linear equation such as (4.1), in which the output, y , is given in terms of its own past values as well as values of the input, is called *recursive*. If past values of y do not appear on the right, then the linear equation is

nonrecursive, that is, recursion is not needed to express y_k in terms of samples of x . Note that (4.1), which is recursive, becomes nonrecursive when expressed in the form of (4.2). However, (4.2) indicates that y_k , in response to a single nonzero input sample x_0 , will continue forever, because $a^k x_0$ is always the final term in the sum. Thus, a recursive linear system is also known generally as an *infinite impulse response (IIR)* system. Conversely, a nonrecursive system is sometimes called a *finite impulse response (FIR)* system.

Another property related to the time-domain behavior of (4.1) is *stability*. A linear system is *stable* if the magnitude of its response to a finite input does not grow without bound. In the case of (4.1), we can see from its nonrecursive form in (4.2), that (4.1) describes a stable system when $|a| < 1$ and an unstable system when $|a| > 1$, because, with $|a| > 1$, the final term in the sum in (4.2), which is $a^k x_0$, increases in magnitude with k for any nonzero value of x_0 . Stability is an important property of linear systems and is discussed later in terms of poles of the transfer function.

In most of our discussions, the weights in the linear equation, that is, b and a in (4.1), are constant real numbers. However, sometimes these weights are equal to, or related to, measured properties of a system, that is, *parameters* of the system, and it is possible that these parameters may change or drift with time, so that the weights are not constant but are functions of k instead. In this case, the linear equation is describing a *time-varying* or *adaptive* signal processing system. Adaptive signal processing, in which the weights change with time and adapt according to a specified purpose, has become an important part of DSP and is introduced in Chapter 9.

The foregoing properties of discrete linear equations are summarized in Table 4.1. For each property, there is a simple example where the property exists and a simple example where the property does not exist.

With these properties in mind, we are ready to discuss linear transfer functions, which, as we shall see, lead to better ways to solve recursive linear equations and, more importantly, provide useful insights into the behavior of linear systems.

TABLE 4.1
Properties of Discrete Linear Systems

Property	Property Exists	Does not Exist
Linear	$y_k = bx_k + ay_{k-1}$	$y_k = b x_k + ay_{k-1}$
Realizable	$y_k = bx_k + ay_{k-1}$	$y_k = bx_k + ay_{k+1}$
Causal	$y_k = bx_k + ay_{k-1}$	$y_k = bx_{k+1} + ay_{k-1}$
Recursive (IIR)	$y_k = bx_k + ay_{k-1}$	$y_k = bx_k + ax_{k-1}$
Stable	$y_k = bx_k + 0.9y_{k-1}$	$y_k = bx_k + 1.1y_{k-1}$
Adaptive	$y_k = b_k x_k + ay_{k-1}$	$y_k = bx_k + ay_{k-1}$

4.3 Discrete Convolution

We begin this section with the general form of (4.1), known as the *direct form*, that describes any causal, realizable linear system with constant weights:

$$y_k = \sum_{n=0}^{N-1} b_n x_{k-n} - \sum_{m=1}^{M-1} a_m y_{k-m}; \quad k = 0, 1, \dots \quad (4.3)$$

Causality ($n \geq 0$) is not essential in this discussion, but it simplifies the index notation, and it also enables the system to operate in “real time,” which is often required. Also, we can usually define the time index, k , so that the signals are zero for $k < 0$, and we will assume this unless we state otherwise. Thus, we have a *startup* sequence that begins as follows:

$$\begin{aligned} y_0 &= b_0 x_0, \\ y_1 &= b_0 x_1 + b_1 x_0 - a_1 y_0, \\ y_2 &= b_0 x_2 + b_1 x_1 + b_2 x_0 - (a_1 y_1 + a_2 y_0), \dots \text{ etc.} \end{aligned} \quad (4.4)$$

The startup sequence continues until $k = M - 1$ or $k = N - 1$, whichever is later, that is, until the final terms in the sums in (4.3) involve samples of x and y at and above $k = 0$. From this point, the computation in (4.3) involves a constant number of $(N + M - 1)$ products.

Each of the sums in (4.3) is a *convolution* of two vectors, a weight vector and a signal vector. Convolution is a fundamental operation in continuous as well as digital signal processing. An illustration of the left-hand convolution in (4.3), with $N = 4$, is given in Figure 4.1. There is nothing really new in the illustration, but it is often helpful to picture the convolution as

$$\begin{aligned} \begin{bmatrix} & & b_0 & b_1 & b_2 & b_3 \\ \dots & x_4 & x_3 & x_2 & x_1 & x_0 \end{bmatrix} & y_0 = b_0 x_0 \\ \begin{bmatrix} & & b_0 & b_1 & b_2 & b_3 \\ \dots & x_4 & x_3 & x_2 & x_1 & x_0 \end{bmatrix} & y_1 = b_0 x_1 + b_1 x_0 \\ \begin{bmatrix} & & b_0 & b_1 & b_2 & b_3 \\ \dots & x_4 & x_3 & x_2 & x_1 & x_0 \end{bmatrix} & y_2 = b_0 x_2 + b_1 x_1 + b_2 x_0 \\ \begin{bmatrix} & & b_0 & b_1 & b_2 & b_3 \\ \dots & x_{k+1} & x_k & x_{k-1} & x_{k-2} & x_{k-3} \end{bmatrix} & y_k = b_0 x_k + b_1 x_{k-1} + b_2 x_{k-2} + b_3 x_{k-3}; \quad k \geq 3 \end{aligned}$$

FIGURE 4.1

Discrete convolution of signal vector x with weight vector b , showing how the vectors are aligned at each time step to produce the sum of products on the right.

shown, with the signal vector, x , reversed and aligned with the weight vector, b , in accordance with the time index, k , so that each product in the sum is evident.

4.4 The z-Transform and Linear Transfer Functions

Just as we used the discrete Fourier transform (DFT) to transform signals from the time domain to the frequency domain, we may also use the DFT to transform the convolutions in (4.3) from the time domain to the frequency domain. To do this, we define the *z-transform*, which is really just the DFT expressed with modified notation using the substitution

$$z = e^{j\omega T} \quad (4.5)$$

in which ωT is frequency in radians as described in Section 3.6. The z-transform of a vector x , which we denote $X(z)$, is the continuous DFT (3.25) with the substitution (4.5), that is,

$$\text{z-transform: } X(z) = \sum_{n=0}^{N-1} x_n z^{-n} \quad (4.6)$$

The limits of the sum in this definition are not fixed; they are set, in each case, to include all the samples in the vector x , or whatever set of samples we may wish to include, in the transform. Either limit may be infinite if so desired. For example, suppose $x_k = e^{-\alpha k}$ for $k = [0:\infty)$. Then

$$\text{z-transform: } X(z) = \sum_{k=0}^{\infty} e^{-\alpha k} z^{-k} = \frac{z}{z - e^{-\alpha}}; \quad \text{DFT: } X(e^{j\omega T}) = \frac{e^{j\omega T}}{e^{j\omega T} - e^{-\alpha}} \quad (4.7)$$

The closed expression for $X(z)$ is found using the geometric series formula (1.20), which, together with (1.19), are the key formulas used for expressing z-transforms of functions of this sort in closed form. In modern DSP, we deal mostly with sample vectors that are not usually expressible as simple discrete functions such as x_k in (4.7). Thus, in this text, we do not (except in a few special cases) discuss the theory of forward and inverse z-transforms of functions. References¹⁻³ are recommended for this purpose.

We now consider an application of z-transforms, namely, the z-transform of a convolution. The transform of a convolution such as those in (4.3) is

found as follows:

$$\begin{aligned}
 \text{Let } u_k &= \sum_{n=0}^{N-1} b_n x_{k-n}, \quad 0 \leq k < \infty \\
 \text{Then } U(z) &= \sum_{k=0}^{\infty} \sum_{n=0}^{N-1} b_n x_{k-n} z^{-k} = \sum_{n=0}^{N-1} b_n z^{-n} \sum_{k=0}^{\infty} x_{k-n} z^{-(k-n)} \\
 &= \sum_{n=0}^{N-1} b_n z^{-n} \sum_{i=0}^{\infty} x_i z^{-i} = B(z)X(z)
 \end{aligned} \tag{4.8}$$

In the final sum, we note that $i = k - n$, and i can begin at zero, because k begins at zero, and x_k is zero for $k < 0$. Thus, the z -transform of the convolution shown here is the product of the z -transforms of the two vectors being convolved. This is true in all cases in which one of the vectors has infinite extent, as x has in this case. Cases in which both vectors are finite will be discussed later.

With the result in (4.8), we are now able to transform the direct-form equation in (4.3). Let $a_0 \equiv 1$. Then (4.3) and its transform become

$$\begin{aligned}
 \sum_{m=0}^{M-1} a_m y_{k-m} &= \sum_{n=0}^{N-1} b_n x_{k-n}; \quad 0 \leq k < \infty \\
 A(z)Y(z) &= B(z)X(z) \\
 \frac{Y(z)}{X(z)} &= \frac{B(z)}{A(z)} = \frac{b_0 + b_1 z^{-1} + \dots + b_{N-1} z^{-(N-1)}}{1 + a_1 z^{-1} + \dots + a_{M-1} z^{-(M-1)}} \equiv H(z)
 \end{aligned} \tag{4.9}$$

When the transforms are written in this form, the quotient $H(z) = B(z)/A(z)$ is called a *transfer function*, because it contains all the information on how to relate the input signal to the output signal, that is, how to “transfer” the signal through the system. Furthermore, the transfer function supplies this information in the *frequency domain* because of the association of z with frequency in (4.5). In other words, equivalent descriptions of a discrete linear system in the time and frequency domains are as follows:

$$\text{Time domain: } y_k = \sum_{n=0}^{N-1} b_n x_{k-n} - \sum_{m=1}^{M-1} a_m y_{k-m} \tag{4.10}$$

$$\text{Transfer function: } H(z) = \frac{Y(z)}{X(z)} = \frac{b_0 + b_1 z^{-1} + \dots + b_{N-1} z^{-(N-1)}}{1 + a_1 z^{-1} + \dots + a_{M-1} z^{-(M-1)}} \tag{4.11}$$

$$\text{Frequency response: } H(e^{j\omega T}) = \frac{b_0 + b_1 e^{-j\omega T} + \dots + b_{N-1} e^{-j(N-1)\omega T}}{1 + a_1 e^{-j\omega T} + \dots + a_{M-1} e^{-j(M-1)\omega T}} \tag{4.12}$$

Now, $H(e^{j\omega T})$ is, as we have said, the ratio of the DFTs of the vectors a and b . Thus, just as the amplitude and phase spectra of a signal vector are given

in (3.18) as the magnitude and angle of the DFT of the vector, so the *amplitude response* (or *amplitude gain*) and *phase response* (or *phase shift*) of a linear system are given by the magnitude and angle of $H(e^{j\omega T})$ in (4.12); that is,

$$\text{Amplitude gain} = \text{abs}(H(e^{j\omega T})) = \frac{|B(e^{j\omega T})|}{|A(e^{j\omega T})|} \quad (4.13)$$

$$\text{Phase shift} = \text{angle}(H(e^{j\omega T})) = \angle B(e^{j\omega T}) - \angle A(e^{j\omega T})$$

An example expressed in MATLAB notation of the computation of (4.13) is given next in (4.14). The weight vectors, a and b , with $M = N = 5$ in (4.3), are copied into the first two expressions. These were generated by a function to be discussed in Chapter 6. So, for the present, we take b and a as given:

```
b=[0.2021 0.3328 0.4517 0.3328 0.2021];
a=[1.0000 -0.2251 0.6568 0.0349 0.0548];
H=fft(b,1000)./fft(a,1000);
subplot(1,2,1);
plot(linspace(0,.5,501),abs(H(1:501)));
subplot(1,2,2);
plot(linspace(0,.5,501),unwrap(angle(H(1:501))));
```

(4.14)

Next, $H(e^{j\omega T})$ is computed as the ratio of DFTs in (4.12). Both DFTs are extended with zeros to length 1000 to produce a more densely-packed response vector, and thus allow continuous amplitude gain and phase plots. (See "Zero Extension" Chapter 3, Table 3.3.) Amplitude and phase response vectors are then computed as in (4.13). [The operation of *unwrap* was explained in connection with (3.21) in Chapter 3.] The response curves are plotted in the two subplots in Figure 4.2. Because, as explained in Chapter 3, the second half of a DFT is a mirror image if the first half, it is conventional to plot responses only up to half the sampling rate, that is, only for $\omega = [0, \pi/T]$ rad/s, or $\nu = [0, 0.5]$ Hz-s. Expressions for labels, and so forth, in the figure are not shown in (4.14) for the sake of simplicity. The complete m-file is included in the software that comes with this text.

As a matter of interest, the amplitude gain curve in Figure 4.2 is called a "lowpass" characteristic, because it describes a system that passes low-frequency components and blocks high-frequency components of a signal.

4.5 Poles and Zeros

There is a geometrical interpretation of the transfer function, $H(z)$, which is a useful aid in understanding the steady-state frequency response and also the transient response characteristics of a linear system. The interpretation involves the *z-plane*, with Cartesian coordinates that are the real and

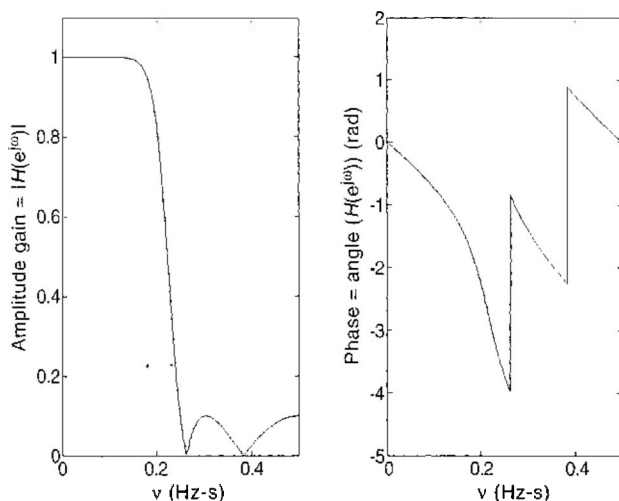


FIGURE 4.2

Amplitude gain and phase response of the linear system with weights given in (4.14). The system is called a "lowpass filter," because it passes the low-frequency components of a signal and blocks the high-frequency components.

imaginary parts of z , and the *poles* and *zeros* of $H(z)$. The latter are the roots, respectively, of the denominator and numerator of $H(z)$ in (4.11).

We can best explain this transfer-function geometry with a simple example. Suppose the linear transfer function, corresponding with $y_k = x_k + x_{k-1} + 0.9y_{k-1}$, is as follows:

$$H(z) = \frac{1 + z^{-1}}{1 - 0.9z^{-1}} = \frac{z + 1}{z - 0.9} \quad (4.15)$$

The pole at $z = 0.9$ and the zero at $z = -1$ are plotted on the z -plane in the left half of Figure 4.3. The numerator and denominator of the complex gain, $H(e^{j\omega T})$, are also shown as vectors on the z -plane. First, we see that the *amplitude gain* is a ratio of *distances*, α/β ; that is,

$$|H(e^{j\omega T})| = \frac{|e^{j\omega T} + 1|}{|e^{j\omega T} - 0.9|} = \frac{\alpha}{\beta} \quad (4.16)$$

The point on the unit circle labeled $e^{j\omega T}$ designates the frequency at which the gain is being observed and is called the *operating point*. As the frequency, ω rad/s, moves from zero to half the sampling rate, the operating point moves from 0 to 180° around the upper half of the unit circle. As it moves, we can see, in this case, that α decreases from 2.0 to 0, and β increases from 0.1 to 1.9. Thus, we are able to see, from the locations of the pole and zero, the form of

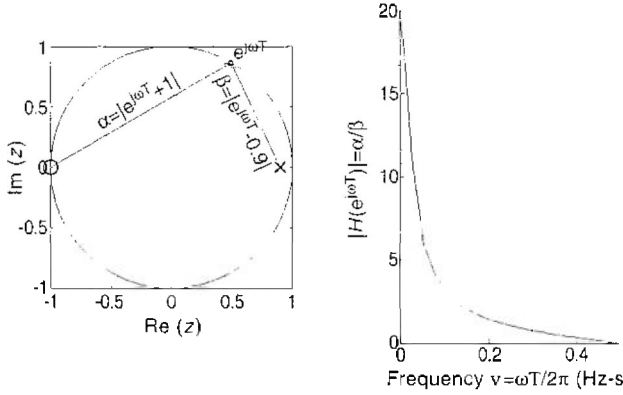


FIGURE 4.3 (Left) Pole-zero plot of $H(z) = (z + 1)/(z - 0.9)$, showing how the amplitude gain at ω rad/s may be visualized as a quotient of lengths; that is, $|H(e^{j\omega T})| = \alpha/\beta$. (Right) Plot of $|H(e^{j\omega T})|$ versus frequency in Hz-s.

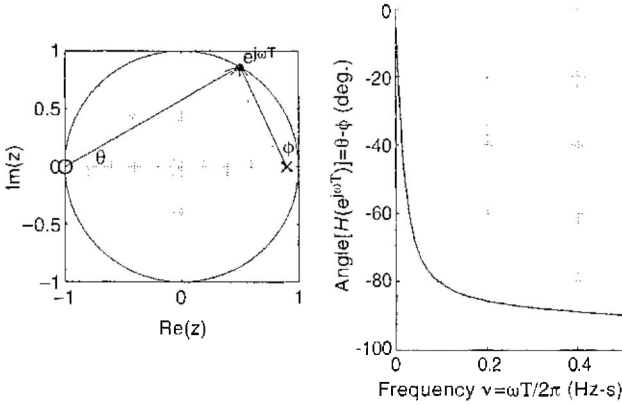


FIGURE 4.4 (Left) Pole-zero plot of $H(z) = (z + 1)/(z - 0.9)$, showing how the phase shift at ω rad/s may be visualized as a difference of angles; that is, $\angle H(e^{j\omega T}) = \theta - \phi$. (Right) Plot of $\angle H(e^{j\omega T})$ versus frequency in Hz-s.

the amplitude gain plot, without actually calculating values in (4.16). The amplitude gain plot is shown in the right half of Figure 4.3.

A similar plot of the *phase response* of the linear system in (4.15) is shown on the left in Figure 4.4. As in (4.13), the phase response is the *algebraic sum of angles* of the poles and zeros of $H(e^{j\omega T})$. In Figure 4.4, the zero angle is labeled θ , and the pole angle is labeled ϕ , so the phase response is as follows:

$$\angle H(e^{j\omega T}) = \theta - \phi \tag{4.17}$$

Notice that as ω increases from 0 to π , $[\theta, \phi]$ increase from $[0, 0]$ to $[\pi/2, \pi]$. Thus, the phase response in (4.17) decreases from 0 to $-\pi/2$, as seen on the right in Figure 4.4.

More generally, to find the poles and zeros of a linear system in the form of (4.11), the first step is to write $H(z)$ as a ratio of polynomials in z :

$$H(z) = \frac{b_0 + b_1 z^{-1} + \dots + b_{N-1} z^{-(N-1)}}{1 + a_1 z^{-1} + \dots + a_M z^{-(M-1)}} = z^{M-N} \frac{b_0 z^{N-1} + b_1 z^{N-2} + \dots + b_{N-1}}{z^{M-1} + a_1 z^{M-2} + \dots + a_{M-1}} \quad (4.18)$$

Then, the poles and zeros are the roots of the denominator and numerator polynomials in (4.18). Notice that, if $N \neq M$, there is also a pole or zero of order $|M - N|$ at $z = 0$.

In MATLAB, the roots may be determined with the *roots* function. The syntax is $r = \text{roots}(w)$, in which w is a vector of weights, and r is a vector of roots. The corresponding polynomial form for $r = \text{roots}(w)$ is as follows:

$$p(z) = w_1 z^{N-1} + w_2 z^{N-2} + \dots + w_N = w_1 (z - r_1)(z - r_2) \dots (z - r_{N-1}) \quad (4.19)$$

The *roots* function allows us to make pole-zero plots easily using MATLAB. For example, the system in (4.14) resulted in the gain and phase plots in Figure 4.2. The poles and zeros of the transfer function may be obtained with the following expressions:

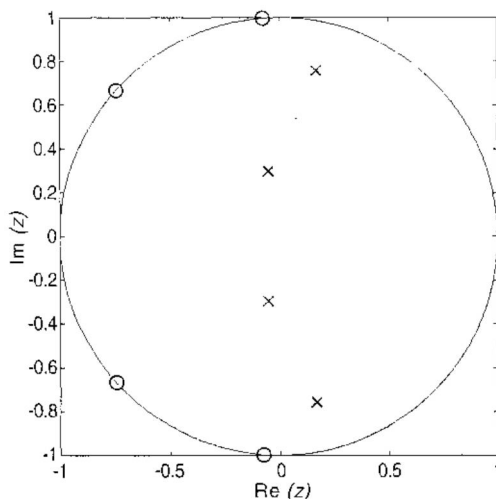
<pre>b=[0.2021 0.3328 0.4517 0.3328 0.2021]; a=[1.0000 -0.2251 0.6568 0.0349 0.0548]; zeros=roots(b) poles=roots(a)</pre>	(4.20)
--	--------

When these expressions are executed, the zeros and poles are printed in the MATLAB command window as follows:

zeros=	poles=	
-0.7444 + 0.6677i	0.1668 + 0.7577i	(4.21)
-0.7444 - 0.6677i	0.1668 - 0.7577i	
-0.0789 + 0.9969i	-0.0542 + 0.2968i	
-0.0789 - 0.9969i	-0.0542 - 0.2968i	

When roots are complex, as these are, they always appear in *conjugate pairs*. Otherwise, as one can easily see by reconstructing the polynomial from the roots, some of the weights would have to be complex.

The poles and zeros in (4.21) are plotted on the z -plane in Figure 4.5. Imagine moving the operating point around the unit circle from $\omega T = 0$ to $\omega T = \pi$, and note how the ratio of distances from the operating point to all the poles and zeros produces the amplitude gain plot in Figure 4.2.

**FIGURE 4.5**

Poles (x) and zeros (o) of the linear transfer function with coefficients given in (4.14) and amplitude and phase responses plotted in Figure 4.2.

Also note that the 180° -phase jumps produced by the zeros on the unit circle in Figure 4.5 appear on the phase plot in Figure 4.2. In this way, the pole-zero plot illustrates the response of a linear system at all frequencies.

The function `pz_plot(b, a)` is included with this text. It uses the `roots` function internally to produce the actual plot in a MATLAB figure window. The plot in Figure 4.5 was produced by executing `pz_plot(b, a)`, where *b* and *a* were specified as in (4.20).

4.6 Transient Response and Stability

Having the poles and zeros of a linear system allows us to express the direct-form transfer function in (4.11) as a *sum of partial fractions*. Even with the poles and zeros given, the partial-fraction form is not easy to construct. Another MATLAB function called `residue` is useful in the construction, and it is, in fact, used in the function `part_frac_exp` provided with this text, which computes the pole-zero expansion of the direct-form transfer function (4.11). The operation of these functions is complicated when there are *multiple poles* [a multiple pole is due to a term of the form $(z - p)^n$, with $n > 1$ in the denominator of $H(z)$]. We discuss the partial fraction form only briefly in this section, because normally, in DSP, partial-fraction expansions of transfer functions are needed only in the special situations discussed here.

For an example of the use of *part_frac_exp*, we again use the linear system with weights given in (4.14) as follows:

$$\begin{aligned} b &= [0.2021 \quad 0.3328 \quad 0.4517 \quad 0.3328 \quad 0.2021]; \\ a &= [1.0000 \quad -0.2251 \quad 0.6568 \quad 0.0349 \quad 0.0548]; \\ [r, p, c] &= \text{part_frac_exp}(b, a) \end{aligned} \quad (4.22)$$

When these expressions are executed, the result is

$$\begin{array}{ccc} r = & p = & c = \\ \begin{array}{l} 1.9720 + 5.3690i \\ 1.9720 - 5.3690i \\ -0.1099 + 0.3623i \\ -0.1099 - 0.3623i \end{array} & \begin{array}{l} -0.0542 - 0.2968i \\ -0.0542 + 0.2968i \\ 0.1668 - 0.7577i \\ 0.1668 + 0.7577i \end{array} & 3.6880 \end{array} \quad (4.23)$$

And, the corresponding partial-fraction form of $H(z)$ is

$$H(z) = c + \frac{r_1 p_1 z}{z - p_1} + \frac{r_2 p_2 z}{z - p_2} + \frac{r_3 p_3 z}{z - p_3} + \frac{r_4 p_4 z}{z - p_4} \quad (4.24)$$

Further aspects of partial fraction expansions may be found in the references,^{1,2,12} but we wish to turn to a more important topic, namely, *transient response* and the question of *stability*.

The *transient response* of a system is the output signal resulting from a single transient input signal, such as an impulse or step function. In particular, the *impulse response* of a linear system is the response to a single unit sample at $t = 0$, that is, the response to the *unit impulse*:

$$\text{Unit impulse: } i = [1 \ 0 \ 0 \ \dots] \quad (4.25)$$

In definition (4.6), we can see that the z -transform of the unit impulse is $I(z) = 1$, and therefore, the impulse response of any linear system is just the inverse transform of the transfer function, that is, $Z^{-1}\{H(z)\}$.

Consider now the impulse response of $H(z)$ in (4.24), which is a constant, c , plus the sum of responses contributed by each additional term. Any of these terms describes a simple linear system of the form given in (4.1), with $b = r_n p_n$, and $a = p_n$ and the corresponding "solution" for the output in (4.2). When applying this solution with $x = i$ in (4.25), the impulse response contribution from this n th term is found to be of the following form:

$$y_k = r_n p_n (p_n)^k = r_n p_n^{k+1}; \quad k = 0, 1, \dots \quad (4.26)$$

Here, we can see that the impulse response will increase with k , provided $|p| > 1$, that is, provided the pole is outside the unit circle on the z -plane,

$|z| = 1$. From this result, we can easily reach the following general conclusion concerning the stability of linear systems:

A system is *unstable* if its response to a transient input increases without bound. The linear system described by $H(z)$ is stable if and only if all poles of $H(z)$ are inside the unit circle on the z -plane. (4.27)

This is the usual criterion for stability. If $H(z)$ has pole(s) *on* and inside the unit circle, the system is said to be *conditionally stable*.

4.7 System Response via the Inverse z-Transform

We have just seen a situation where the inverse z -transform was useful in determining the response of a linear system. In general, given analytic descriptions of the input signal, x , and the transfer function, $H(z)$, we can obtain the output signal, y , as an inverse z -transform, that is,

$$y = Z^{-1}\{X(z)H(z)\} \quad (4.28)$$

The uses of this approach are limited, because signals are not usually known in analytic form, but sometimes, the impulse or step-function response of a system, or the transient response to a sinusoidal input, is required, and the inverse transform approach is useful in these cases.

In general, inverse z -transforms and inverse Laplace transforms are derived with the help of the Residue Theorem, which is part of complex variable theory.⁴ Because this subject is not a normal prerequisite for a first course in DSP, we will not pursue it here. Instead, the list of z -transforms in Table 4.2 is provided. The table is arranged in conventional form, with the transforms on the right.

By combining lines in the table, one can find transforms and inverse transforms of a wide variety of functions. The lettered lines, A through E, are functional relations that may be proved from the definition of the z -transform, which appears on line A. Notice that lines B and C are equivalent to the linearity and phase shift properties (lines 2 and 8) of the DFT in Table 3.3.

The numbered lines, 1 through 7, then provide the transforms of specific functions. These lines may be combined or modified using the relations on the lettered lines to produce other transform pairs. For example, we would produce the transform of $e^{-\lambda k} \sin(\alpha k)$ using lines D and 4 of Table 4.2 as follows:

$$Z\{e^{-\lambda k} \sin(\alpha k)\} = \frac{ze^{\lambda} \sin \alpha}{z^2 e^{2\lambda} - 2ze^{\lambda} \cos \alpha + 1} \quad (4.29)$$

TABLE 4.2

A Short Table of z-Transforms

Line	Function of k	Function of z
A	$x = [x_0, x_1, \dots]$	$X(z) = \sum_{k=0}^{\infty} x_k z^{-k}$
B	$\alpha x + \beta y$	$\alpha X(z) + \beta Y(z)$
C	$y_k = \begin{cases} 0; & k < \lambda \\ x_{k-\lambda}; & k \geq \lambda \end{cases}$	$Y(z) = z^{-\lambda} X(z)$
D	$y_k = e^{-\alpha k} x_k; k \geq 0$	$Y(z) = X(ze^{\alpha})$
E	$y_k = kx_k; k \geq 0$	$Y(z) = -z \frac{\partial}{\partial z} X(z)$
1	$[1, 0, 0, \dots]$ (impulse function)	1
2	$[1, 1, 1, \dots]$ (step function)	$\frac{z}{z-1}$
3	$\alpha^k; k \geq 0$ and $ \alpha < 1$	$\frac{z}{z-\alpha}$
4	$\sin(\alpha k); k \geq 0$	$\frac{z \sin \alpha}{z^2 - 2z \cos \alpha + 1}$
5	$\cos(\alpha k); k \geq 0$	$\frac{z(z - \cos \alpha)}{z^2 - 2z \cos \alpha + 1}$
6	$\frac{\alpha^k - \beta^k}{\alpha - \beta}; \alpha , \beta < 1$	$\frac{z}{(z-\alpha)(z-\beta)}$
7	$\frac{R^k}{\beta} \sin(k\theta); R = \sqrt{\alpha^2 + \beta^2}, \theta = \tan^{-1}\left(\frac{\beta}{\alpha}\right)$	$\frac{z}{(z-\alpha)^2 + \beta^2}$

Table 4.2 may be used in a number of ways to determine transient response characteristics of linear systems. For example, we saw in the previous section that a linear transfer function may be expressed as a sum of partial fractions, as in (4.24). The conjugate pairs of fractions may then be combined to produce rational quadratic forms with real coefficients. We will discuss this form of the transfer function (called the *parallel form*) in a later section, but here we note that with $H(z)$ written in the parallel form, lines 6 and 7 of Table 4.2 may be used to determine the impulse response of each term, and thus (using line B), the impulse response of $H(z)$. There are exercises on this and other topics relating to Table 4.2 at the end of this chapter.

4.8 Cascade, Parallel, and Feedback Structures

We have seen how the linear transfer function, $H(z)$ in (4.11), may be factored so that its numerator and denominator are in the form of (4.19) or are written as a sum of partial fractions in the form of (4.24). The terms in the factored version may be combined to form a *cascade filter structure*, and the terms in the partial-fraction version may be combined to form a *parallel structure*.

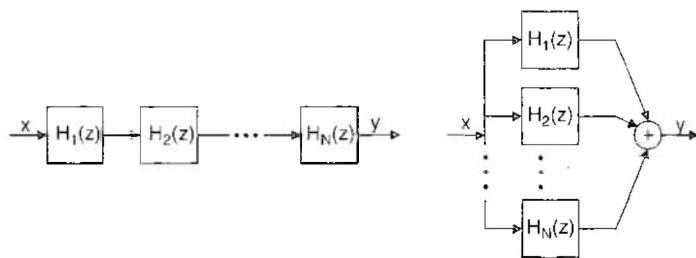


FIGURE 4.6
Cascade (left) and parallel (right) filter structures.

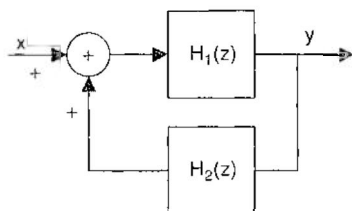


FIGURE 4.7
Linear feedback. The output, y , is filtered by $H_2(z)$, and the result is added to the input, x . Then the sum is filtered by $H_1(z)$ to produce y . For realizability, the path through $H_1(z)$ and $H_2(z)$ must include a delay.

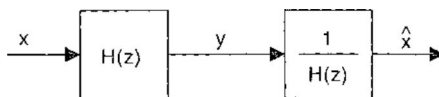
Both structures are illustrated in Figure 4.6. The corresponding relationships are as follows:

$$\text{Cascade: } H(z) = \prod_{n=1}^N H_n(z); \quad \text{Parallel: } H(z) = \sum_{n=1}^N H_n(z) \quad (4.30)$$

When the two forms are used to implement a given linear transfer function, $H(z)$, the overall transfer function is the same, but the individual terms, $H_n(z)$, are not the same. They imply different computations in the time domain, and sometimes, especially in DSP hardware design, one form may be preferred.

A third form, the *feedback structure*, is illustrated in Figure 4.7. The output signal, y , is processed by $H_2(z)$ and added to the input signal, x . The sum is then processed by $H_1(z)$ to produce y . For realizability, element y_k obviously cannot be computed in terms of itself. Therefore, we must include a delay in the path through $H_1(z)$ and $H_2(z)$. Assuming this is true and the structure is realizable, we can apply the transform relation (4.11) and obtain

$$Y(z) = (X(z) + Y(z)H_2(z))H_1(z)$$

**FIGURE 4.8**

Inverse operation to recover x from y .

Solving this for the overall transfer function, $H(z)$, we obtain

$$H(z) = \frac{Y(z)}{X(z)} = \frac{H_1(z)}{1 - H_1(z)H_2(z)} \quad (4.31)$$

The stability of the linear feedback form does not depend on the individual stabilities of $H_1(z)$ and $H_2(z)$, but instead relies on the location of the poles in (4.31). Notice also, that for $H(z)$ to be *realizable*, the combination $H_1(z)H_2(z)$ must delay the input by at least one sample.

The final form we wish to discuss in this section, illustrated in Figure 4.8, involves a simple concept—the *inversion* of a transfer function. In the figure, we can see that if the second system is the inverse of the first, then, except for delays and possible roundoff errors, vectors $\hat{x}$ and x should be identical. Two questions arise about such a system: (1) is it ever useful, and (2) when is the inverse operation realizable?

Regarding (1), there are at least two applications—encryption and compression. If x is a secret signal, then $H(z)$ could amount to an “encryption key” that allows one to send y in the clear and allows a receiver with knowledge of $H(z)$ to recover x from y . In the compression application, $H(z)$ could be a filter that reduces the amount of information in x in a manner such that y requires a shorter description and is thus a *compressed* version of x . There is an example of signal compression and recovery in Chapter 8.

The answer to the second question is that, at least with signal vectors of finite length, inversion is essentially always possible. This can be seen simply by solving (4.9) for x_k :

$$x_k = \frac{1}{b_0} \left[y_k + \sum_{m=1}^{M-1} a_m y_{k-m} - \sum_{n=1}^{N-1} b_n x_{k-n} \right]; \quad k \geq 0 \quad (4.32)$$

In accordance with Table 4.1, this result is both realizable and causal. One may argue further that because y was computed originally from x , and y is finite, instability cannot become a problem in this case.

As a final point, (4.32) is obviously not valid if $b_0 = 0$. If such is the case, then (4.9) can be solved for x_{k-i} , where b_i is the first nonzero weight. Then, the inverse operation involves a delay but is still realizable.

The structures reviewed in this section are representative of a variety of electromechanical systems and processes, as well as other physical phenomena. The feedback concept is applicable to *feedback control systems*, that is, control systems that measure their own performance. Many control applications

that were formerly accomplished with continuous electromechanical systems are now implemented with digital processors, essentially in the form of Figure 4.7.

4.9 Direct Algorithms

Each of the structures just described is an assembly of linear transfer functions; each is in the direct form of (4.11). The direct form of the transfer function implies a computational procedure, or *algorithm*, in the time domain, just as (4.11) implies (4.10). But, (4.10) represents only one of several algorithms, all of which produce (4.11) as the transfer function. The algorithms are all equivalent in that they produce the same output signal in response to a given input signal, but each implies a different set of computations, or, if implemented in hardware, a different chip design.

First, three *direct* forms of (4.10) are shown in Figures 4.9, 4.10, and 4.11. These are all equivalent and produce the same output for a given input, but the algorithms differ and imply different realizations in hardware. For example, the first diagram would require twice as many storage units compared with the other two. In languages like MATLAB, any of the three forms may easily be implemented. The MATLAB *filter* function accepts data in the form of Figure 4.9.

The diagram in Figure 4.9 is seen to represent explicitly the direct form in (4.10). The “*u*” symbols in Figures 4.10 and 4.11 each stand for intermediate signal vectors like *x* and *y*. To prove the form in Figure 4.10 is equivalent to

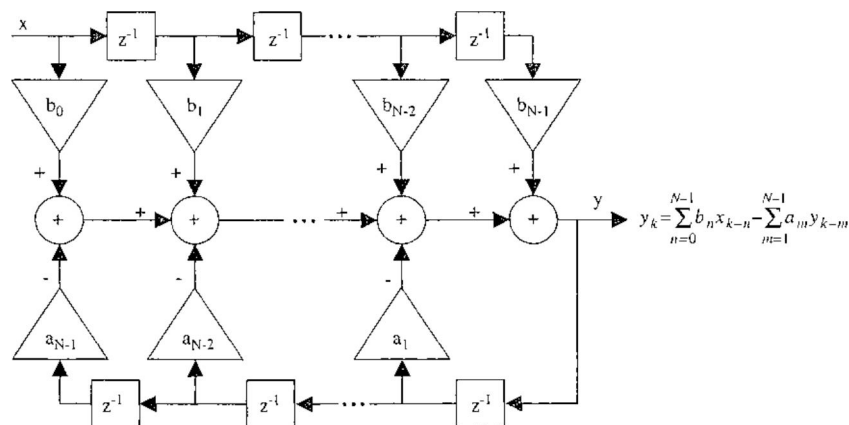


FIGURE 4.9

Linear system diagram 1. The diagram illustrates the direct form in (4.10), with $N = M$. Any of the weights may be zero.

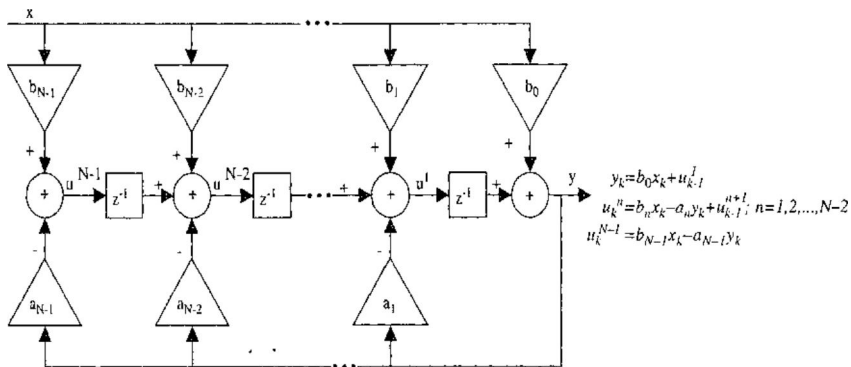


FIGURE 4.10

Linear system diagram 2. The diagram illustrates the direct form with $N = M$, but the form of the algorithm differs from (4.11). Any of the weights may be zero.

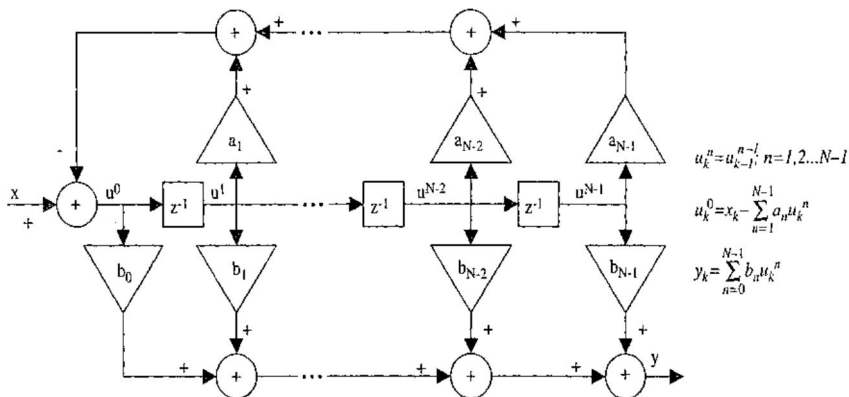


FIGURE 4.11

Linear system diagram 3. The diagram illustrates the direct form with $N = M$, in a form that differs from the first two diagrams. Any of the weights may be zero.

(4.10), substitute the second line of the equation into the first line recursively for $n = 1, 2, \dots, N-2$, and substitute the last line for $n = N-1$, as follows:

$$\begin{aligned}
 y_k &= b_0 x_k + (b_1 x_{k-1} - a_1 y_{k-1} + (b_2 x_{k-2} - a_2 y_{k-2} + \dots + (b_{N-1} x_{k-N+1} - a_{N-1} y_{k-N+1}) \dots)) \\
 &= \sum_{n=0}^{N-1} b_n x_{k-n} - \sum_{m=1}^{N-1} a_m y_{k-m}
 \end{aligned} \quad (4.33)$$

Proving that Figure 4.11 is equivalent to (4.10) is more difficult. The first equation recurses by self-substitution to $u_k^n = u_{k-n}^0$. Substituting this result

into the second equation, we obtain the following:

$$u_k^0 = x_k - \sum_{n=1}^{N-1} a_n u_{k-n}^0, \quad \text{or} \quad x_k = \sum_{n=0}^{N-1} a_n u_{k-n}^0; \quad a_0 \equiv 1 \quad (4.34)$$

Similarly, substituting $u_k^n = u_{k-n}^0$ into the third equation produces

$$y_k = \sum_{n=0}^{N-1} b_n u_{k-n}^0 \quad (4.35)$$

Next, as in (4.8), we take the z -transforms of the convolutions in (4.34) and (4.35) and eliminate the transform of u^0 as follows:

$$X(z) = A(z)U^0(z); \quad Y(z) = BU^0(z); \quad \therefore \frac{Y(z)}{X(z)} = \frac{B(z)}{A(z)} \quad (4.36)$$

This is the transfer function in (4.11), and therefore, Figures 4.11 and 4.9 are equivalent.

4.10 State-Space Algorithms

In addition to the three direct forms in Figures 4.9 through 4.11, we have the *state-space* or *state-vector* form, which is closely related to the third form in Figure 4.11. In this form, the internal *state* of the system at time step k is specified by the *state vector*,

$$u_k = [u_k^{N-1} \ u_k^{N-2} \ \dots \ u_k^1]^T \quad (4.37)$$

Each of the elements of the state vector is called a *state variable* of the system. Note that the state vector is a column vector and does not include the input state variable, u^0 . Next, we define the following arrays, each having $N-1$ rows:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & 1 & 0 & \dots & 0 & 0 \\ \vdots & & & & & & \\ 0 & 0 & 0 & 0 & \dots & 0 & 1 \\ -a_{N-1} & -a_{N-2} & -a_{N-3} & -a_{N-4} & \dots & -a_2 & -a_1 \end{bmatrix} \quad B = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix} \quad C = \begin{bmatrix} b_{N-1} - b_0 a_{N-1} \\ b_{N-2} - b_0 a_{N-2} \\ \vdots \\ b_2 - b_0 a_2 \\ b_1 - b_0 a_1 \end{bmatrix} \quad (4.38)$$

With these definitions, the state-space equations for Figure 4.11 are as follows:

$$\boxed{\begin{aligned} u_{k+1} &= A * u_k + B x_k \\ y_k &= C' * u_k + b_0 x_k \end{aligned}} \quad (4.39)$$

These are the state-space equations for a linear discrete system, with array products indicated with " $*$ ". The state-space form is used often in control literature and with feedback systems in general.

To show that the state-space equations represent the diagram in Figure 4.11 and therefore any linear system, we provide the following demonstration with $N = 4$, which should be easier to follow and understand than a formal proof. With $N = 4$ and the definitions just given, we have the following arrays:

$$u_k = \begin{bmatrix} u_k^3 \\ u_k^2 \\ u_k^1 \end{bmatrix} \quad A = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -a_3 & -a_2 & -a_1 \end{bmatrix} \quad B = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \quad C = \begin{bmatrix} b_3 - b_0 a_3 \\ b_2 - b_0 a_2 \\ b_1 - b_0 a_1 \end{bmatrix} \quad (4.40)$$

Substituting (4.40) into the first equation in (4.39), we obtain

$$u_{k+1} = \begin{bmatrix} u_{k+1}^3 \\ u_{k+1}^2 \\ u_{k+1}^1 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ -a_3 & -a_2 & -a_1 \end{bmatrix} * \begin{bmatrix} u_k^3 \\ u_k^2 \\ u_k^1 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} x_k = \begin{bmatrix} u_k^2 \\ u_k^1 \\ x_k - \sum_{n=1}^3 a_n u_k^n \end{bmatrix} = \begin{bmatrix} u_k^2 \\ u_k^1 \\ u_k^0 \end{bmatrix} \quad (4.41)$$

Thus, in (4.41), we have the first two equations in Figure 4.11 with $N = 4$. Substituting (4.40) into the second equation in (4.39), we obtain

$$\begin{aligned} y_k &= [b_3 - b_0 a_3 \quad b_2 - b_0 a_2 \quad b_1 - b_0 a_1] * \begin{bmatrix} u_k^3 \\ u_k^2 \\ u_k^1 \end{bmatrix} + b_0 x_k \\ &= b_0 \left(x_k - \sum_{n=1}^3 a_n u_k^n \right) + b_1 u_k^1 + b_2 u_k^2 + b_3 u_k^3 = \sum_{n=0}^3 b_n u_k^n \end{aligned} \quad (4.42)$$

This completes the demonstration with $N = 4$, because the result in (4.42) is the last line in Figure 4.11. The demonstration works in the same way for any value of N .

The state-space equations (4.39) may also be written in a form that allows an output signal element, y_k , to be computed at any time step nonrecursively, that is, in terms only of the initial state vector and the history of the input signal. This nonrecursive form of (4.39) is as follows:

$$\boxed{\begin{aligned} u_k &= A^k * u_0 + \sum_{m=0}^{k-1} A^m * B x_{k-1-m} \\ y_k &= C' * u_k + b_0 x_k \end{aligned}} \quad (4.43)$$

The second lines in (4.39) and (4.43) are identical. The validity of the first line of (4.43) may be proved by induction. First, with $k = 1$, and writing the first line of (4.39) for k instead of $k + 1$, the two equations for u_k are the same. Now, suppose (4.43) is true for time step k . Then, at step $k + 1$, the first line of (4.43) is as follows:

$$\begin{aligned} u_{k+1} &= A * \left(A^k * u_0 + \sum_{n=1}^k A^{n-1} * Bx_{k-n} \right) + Bx_k \\ &= A * \left(A^k * u_0 + \sum_{m=0}^{k-1} A^m * Bx_{k-m-1} \right) + Bx_k \end{aligned} \quad (4.44)$$

But, if (4.43) is true for k , then the quantity in brackets in (4.44) must be u_k , and so (4.44) is the same as the first line in (4.39). Thus, we have shown that (4.43) is true for $k = 1$. Also, we have shown that if (4.43) is true for k , it must be true for $k + 1$. So, the validity of (4.43) is proved by induction.

4.11 Lattice Algorithms and Structures

The dictionary defines a lattice as an open framework of strips of wood or metal interwoven to form a regular pattern. The lattice form of a linear system gets its name, because if you substitute signal paths for the wood and metal strips, its diagram fits the definition.

Lattice algorithms are like direct algorithms in that both provide implementations of any linear system. As we shall see, converting from direct to lattice form is complicated, and the reader may ask whether it is worth the effort, considering that the lattice does nothing really new. The answer is that it may not be worth the effort, and you may wish to skip this section, unless (1) you may be interested in DSP operations that use the lattice structure for greater stability or a preferable hardware layout or (2) you are a student and your instructor thinks this subject is important. In either case, the result of your study may be beneficial.

A single *symmetric two-multiplier lattice stage* is shown in Figure 4.12. These stages are connected to form lattice structures. The signal vectors, u^n , v^n , etc. are indicated at each stage, with the stages being numbered from right to left. (Note that the superscripts are *not* exponents.) In z -transform notation, we can see the following relationships in Figure 4.12:

$$\begin{aligned} U^n(z) &= U^{n+1}(z) - \kappa_n z^{-1} V^n(z) \\ V^{n+1}(z) &= \kappa_n U^n(z) + z^{-1} V^n(z) \end{aligned} \quad (4.45)$$

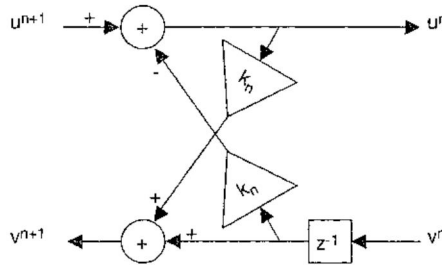


FIGURE 4.12

Symmetric two-multiplier lattice stage.

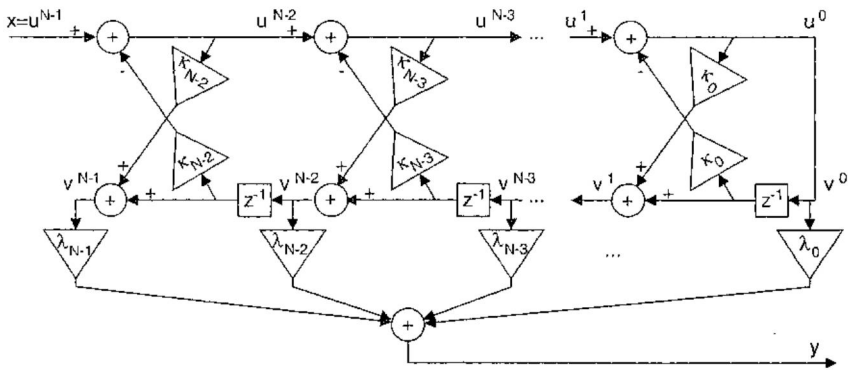


FIGURE 4.13

Symmetric recursive lattice structure.

A form of (4.45) more useful for analysis allows us to move from stage n to stage $n + 1$:

$$\begin{bmatrix} U^{n+1}(z) \\ V^{n+1}(z) \end{bmatrix} = \begin{bmatrix} 1 & \kappa_n z^{-1} \\ \kappa_n & z^{-1} \end{bmatrix} * \begin{bmatrix} U^n(z) \\ V^n(z) \end{bmatrix} \quad (4.46)$$

A lattice may be formed by connecting stages like this in cascade. A *symmetric recursive lattice* equivalent to the direct form is shown in Figure 4.13, following Gray and Markel.⁵ There are $N - 1$ " κ " weights and N " λ " weights; hence, there are the same degrees of freedom as in Figures 4.9 through 4.11. To prove that Figure 4.13 is an implementation of Figure 4.9, and to show how to convert the weights in Figure 4.13 to direct weights, we first normalize (4.46) as follows:

$$P^n(z) = \frac{U^n(z)}{U^0(z)}; \quad Q^n(z) = \frac{V^n(z)}{V^0(z)}; \quad \begin{bmatrix} P^n(z) \\ Q^n(z) \end{bmatrix} = \begin{bmatrix} 1 & \kappa_{n-1} z^{-1} \\ \kappa_{n-1} & z^{-1} \end{bmatrix} * \begin{bmatrix} P^{n-1}(z) \\ Q^{n-1}(z) \end{bmatrix} \quad (4.47)$$

Noting that $u^0 = v^0$ at the right end of Figure 4.13, and therefore, $U^0(z) = V^0(z)$, we can express the transfer function, $H(z)$, in terms of these new variables and the weights in the diagram:

$$\frac{X(z)}{U^0(z)} = P^{N-1}(z); \quad \frac{Y(z)}{U^0(z)} = \sum_{n=0}^{N-1} \lambda_n V^n(z); \quad \therefore H(z) = \frac{Y(z)}{X(z)} = \frac{\sum_{n=0}^{N-1} \lambda_n Q^n(z)}{P^{N-1}(z)} \quad (4.48)$$

Noting that $P^0(z) = Q^0(z) = 1$ by definition, we now have a method for converting the lattice structure in Figure 4.13 to a direct algorithm. The method is summarized as follows:

Lattice-to-Direct Conversion:

$$P^0(z) = Q^0(z) = 1$$

$$\begin{bmatrix} P^n(z) \\ Q^n(z) \end{bmatrix} = \begin{bmatrix} 1 & \kappa_{n-1} z^{-1} \\ \kappa_{n-1} & z^{-1} \end{bmatrix} * \begin{bmatrix} P^{n-1}(z) \\ Q^{n-1}(z) \end{bmatrix}; \quad n = 1, 2, \dots, N-1$$

$$H(z) = \frac{\sum_{n=0}^{N-1} \lambda_n Q^n(z)}{P^{N-1}(z)} = \frac{B(z)}{A(z)}$$

(4.49)

The lattice-to-direct conversion in (4.49) is easy to implement using MATLAB. The function `[b,a] = lat_to_dir(kappa,lambda)` is included with this text for this purpose. The conversion is accomplished with the following expressions within the function:

```
% Initialize P(1:N) and q(1:N) and b(1:N)
% at stage 0.
p=[1,zeros(1,N-1)];
q=[1,zeros(1,N-1)];
b=lambda(1)*q;
% Update p, q, and b at stages 1 thru N-1.
for n=2:N
    p1=p;
    p=p1+kappa(n-1)*[0,q(1:N-1)];
    q=kappa(n-1)*p1+[0,q(1:N-1)];
    b=b+lambda(n)*q;
end
% Set a equal to final value of p(1:N).
a=p;
```

(4.50)

In (4.50), p and q are coefficient vectors of $P^n(z)$ and $Q^n(z)$ in (4.49), and these keep changing in the "for" loop in (4.50). At the n th stage, the vectors are p^n and q^n , and $\lambda_n q^n$ is added to the b vector, that is, the numerator of $H(z)$ in (4.49). At the final stage, $A(z)$ is set equal to $P^{N-1}(z)$, the denominator of $H(z)$ in (4.49).

Having seen equivalent lattice and direct forms, it is reasonable to ask how *stability* translates from direct to lattice. A stable direct system has all poles inside the unit circle, and in (4.49), we can see that the poles of $P^{N-1}(z)$ must also lie inside the unit circle for stability. Suppose $N = 2$, that is, the lattice in Figure 4.13 has only a single stage. Then, in (4.49), $P^{N-1}(z)$ would become $P^1(2)(z) = 1 + \kappa_0 z^{-1}$, and the requirement for stability would be $|\kappa_0| < 1$. Because the lattice stages in Figure 4.13 are cascaded, it is not surprising that, as Jury⁹ has proved, the weight in each stage must satisfy $|\kappa_n| < 1$ for stability. Thus,

For stability in a symmetric lattice:
 $|\kappa_n| < 1$ in each stage of the lattice

We turn now to the question of an inverse for (4.49), that is, a *direct-to-lattice* conversion. First, we prove an interesting property of the symmetric lattice. By repeated use of the second equation in (4.49), we can write

$$\begin{bmatrix} P^n(z) \\ Q^n(z) \end{bmatrix} = \begin{bmatrix} 1 & \kappa_{n-1} z^{-1} \\ \kappa_{n-1} & z^{-1} \end{bmatrix} * \dots * \begin{bmatrix} 1 & \kappa_0 z^{-1} \\ \kappa_0 & z^{-1} \end{bmatrix} * \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad (4.51)$$

[Here, we can see that $p_n^n = \kappa_{n-1}$ and $q_n^n = 1$ are the coefficients of z^{-n} in $P^n(z)$ and $Q^n(z)$, respectively. We will use this property later.] In the first equation in (4.51), we substitute z^{-1} for z and multiply by z^{-n} to obtain

$$\begin{bmatrix} z^{-n} P^n(z^{-1}) \\ Q^n(z) \end{bmatrix} = \begin{bmatrix} z^{-1} & \kappa_{n-1} \\ \kappa_{n-1} & z^{-1} \end{bmatrix} * \dots * \begin{bmatrix} z^{-1} & \kappa_0 \\ \kappa_0 & z^{-1} \end{bmatrix} * \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad (4.52)$$

If we carry out the product in (4.52) from right to left, the elements of the right-hand column vector are always equal. Thus, we have this important relationship at each stage of a symmetric lattice: The coefficient vector q^n is the reverse of the coefficient vector p^n ; that is,

$$\boxed{Q^n(z) = z^{-n} P^n(z^{-1})} \quad (4.53)$$

With this relationship, we can construct a direct-to-lattice algorithm. We rewrite the center equation in (4.49) as follows:

$$\begin{bmatrix} P^n(z) \\ Q^n(z) \end{bmatrix} = \begin{bmatrix} 1 & \kappa_{n-1} \\ \kappa_{n-1} & 1 \end{bmatrix} * \begin{bmatrix} P^{n-1}(z) \\ z^{-1} Q^{n-1}(z) \end{bmatrix} \quad (4.54)$$

Next, we invert the weight matrix in accordance with (1.15) and rewrite (4.54) as follows:

$$\begin{bmatrix} P^{n-1}(z) \\ z^{-1}Q^{n-1}(z) \end{bmatrix} = \frac{1}{1 - \kappa_{n-1}^2} \begin{bmatrix} 1 & -\kappa_{n-1} \\ -\kappa_{n-1} & 1 \end{bmatrix} \begin{bmatrix} P^n(z) \\ Q^n(z) \end{bmatrix} \quad (4.55)$$

The first of these two equations gives us the formula for $P^{n-1}(z)$ in terms of $P^n(z)$:

$$P^{n-1}(z) = \frac{P^n(z) - \kappa_{n-1}Q^n(z)}{1 - \kappa_{n-1}^2}; \quad n = N-1, N-2, \dots, 1 \quad (4.56)$$

Because, as seen in (4.51), $p_n^n = \kappa_{n-1}$ is the coefficient of z^{-n} in $P^n(z)$, we have in (4.56) a method for regressing from left to right in Figure 4.13 and computing κ_{n-1} , p^n , and q^n at each stage. Furthermore, in the numerator of $H(z)$ in (4.49), we can see that λ_{N-1} must be the coefficient of $z^{-(N-1)}$ in the sum, because, again as in (4.51), the coefficient q_{N-1}^{N-1} equals one. Therefore, $\lambda_{N-1} = b_{N-1}$, and we can remove the final term in the sum and thus move to the left in Figure 4.13, computing λ_n similarly at each stage. All of this is summarized in the algorithm (4.57), in which $S^n(z)$ is the reduced numerator of (4.49) at the n th stage.

As before, the conversion in (4.57) from direct to lattice weights is easy to accomplish in MATLAB. The function `[kappa,lambda] = dir_to_lat(b,a)`, which is the inverse of `lat_to_dir`, is included with this text. Within the function, the conversion is accomplished with the expressions in (4.58). The expressions are essentially an implementation of (4.57). The p , q , and s vectors are coefficient vectors of $P^n(z)$, $Q^n(z)$, and $S^n(z)$, and these change during execution. The only difference is that in the m-file, array indices must begin at one instead of zero.

$$\left. \begin{array}{l} \text{Direct -to-Lattice Conversion:} \\ P^{N-1}(z) = A(z); \quad S^{N-1}(z) = B(z); \quad \lambda_{N-1} = b_{N-1} \\ \kappa_{n-1} = p_n^n \\ Q^n(z) = z^{-n}P^n(z) \\ P^{n-1}(z) = \frac{P^n(z) - \kappa_{n-1}Q^n(z)}{1 - \kappa_{n-1}^2}; \quad n = N-1, N-2, \dots, 1 \\ S^{n-1}(z) = S^n(z) - \lambda_n Q^n(z) \\ \lambda_{n-1} = s_{n-1}^{n-1} \end{array} \right\} \quad (4.57)$$

```

% Initialize p, s, kappa, and lambda.
p=a;
s=b;
kappa=zeros(1,N-1);
lambda=[zeros(1,N-1), b(N)];
% Update kappa, q, p, s, and lambda recursively.
for n=N:-1:2
    kappa(n-1)=p(n);
    q=p(n:-1:1);
    p=(p(1:n) - kappa(n-1)*q) / (1 - kappa(n-1)^2);
    s=s(1:n) - lambda(n)*q;
    lambda(n-1)=s(n-1);
end

```

(4.58)

Thus, we have algorithms for transforming from direct to lattice form and vice versa. Lattice filtering is accomplished simply by implementing Figure 4.13, by setting the appropriate initial conditions including $u^{N-1} = x$, where x is the input signal vector, computing successive values of $U^n(z)$ and $V^n(z)$ at each time step in accordance with (4.45) going from left to right in Figure 4.13, and computing y_k at each time step as the sum shown in Figure 4.13. The lattice algorithm may be summarized in z-transform form as follows, with zero initial conditions assumed. The input and output signal vectors in Figure 4.13 are x_k and y_k with $k = 0, 1, \dots, K-1$:

Lattice Filtering:

$$U^{N-1}(z) = X(z)$$

$$\left. \begin{aligned} U^n(z) &= U^{n+1}(z) - \kappa_n V^n(z) \\ V^{n+1}(z) &= \kappa_n U^n + V^n(z) \end{aligned} \right\}; \quad n = N-2, N-3, \dots, 0$$

$$Y(z) = \sum_{n=0}^{N-1} \lambda_n V^n(z)$$

(4.59)

The *lat_filter* function is an implementation of (4.59). The reader may examine the m-file, which is quite short, to see the procedure in detail.

We have seen that a *nonrecursive direct* form could be constructed simply by setting all the “ a ” weights in (4.10) to zero, so $H(z) = B(z)$, a polynomial in z . A *nonrecursive symmetric lattice* form is also possible but is not attained quite as simply. In (4.49), if we set $\lambda_0 = 1$ and the rest of the λ ’s to zero, then, because $Q^0(z) = 1$, we have

$$H(z) = \frac{Y(z)}{X(z)} = \frac{1}{P^{N-1}(z)} = \frac{U^0(z)}{U^{N-1}(z)} \quad (4.60)$$

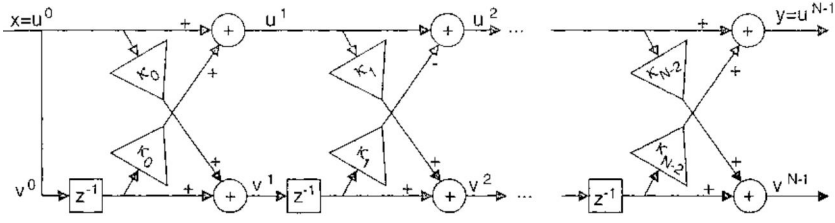


FIGURE 4.14

Symmetric nonrecursive lattice structure.

Now suppose in (4.60) that we could reverse the signal flow and thus invert the transfer function. Then, we would have a nonrecursive filter with transfer function:

$$H(z) = P^{N-1}(z) = \frac{U^{N-1}(z)}{U^0(z)} = B(z) = 1 + b_1 z^{-1} + \dots + b_{N-1} z^{-(N-1)} \quad (4.61)$$

That is, a polynomial in z with $b_0 = 1$. But, this requires $X(z) = U^0(z)$ and $Y(z) = U^{N-1}(z)$, which means that the signal in each lattice stage must flow from u^n to u^{n+1} , *without changing the relationships* in (4.45). Thus, the operation of the lattice stages must be described by (4.46) instead of (4.45). A lattice of this form is shown in Figure 4.14. The lattice-to-direct conversion is the same as in (4.49), except now $H(z) = P^{N-1}(z)$:

Nonrecursive Lattice-to-Direct Conversion:

$$P^0(z) = Q^0(z) = 1$$

$$\begin{bmatrix} P^n(z) \\ Q^n(z) \end{bmatrix} = \begin{bmatrix} 1 & \kappa_{n-1} z^{-1} \\ \kappa_{n-1} & z^{-1} \end{bmatrix} * \begin{bmatrix} P^{n-1}(z) \\ Q^{n-1}(z) \end{bmatrix}; \quad n = 1, 2, \dots, N-1 \quad (4.62)$$

$$H(z) = P^{N-1}(z)$$

Similarly, the direct-to-lattice conversion algorithm follows from (4.57) with $P^{N-1}(z)$ initialized to $B(z)$ to conform with (4.61):

Nonrecursive Direct-to-Lattice Conversion:

$$P^{N-1}(z) = B(z) = 1 + b_1 z^{-1} + \dots + b_{N-1} z^{-(N-1)}$$

$$\left. \begin{aligned} \kappa_{n-1} &= p_n \\ Q^n(z) &= z^{-n} P^n(z) \\ P^{n-1}(z) &= \frac{P^n(z) - \kappa_{n-1} Q^n(z)}{1 - \kappa_{n-1}^2} \end{aligned} \right\}; \quad n = N-1, N-2, \dots, 1 \quad (4.63)$$

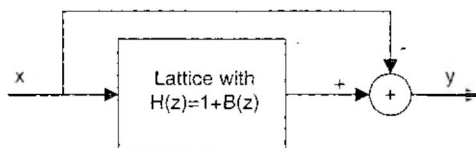


FIGURE 4.15

Conversion to nonrecursive lattice when $b_0 \neq 1$. The overall transfer function is $Y(z)/X(z) = B(z)$.

Note that the direct weight b_0 must equal one. If this is not so, that is, $b_0 \neq 1$ in (4.63), one might first think of dividing $B(z)$ by b_0 , but if $b_{N-1} = b_0$ (and this is often the case as we will see in the next chapter), then dividing by b_0 would result in $\kappa_{N-1} = 1$, and again, (4.63) would not work. So, the preferred approach when $b_0 \neq 1$ is to convert $H(z) = 1 + B(z)$ to a lattice using (4.63), then subtract one from the transfer function by subtracting the input from the output as in Figure 4.15.

A MATLAB function called `nr_dir_to_lat` converts any causal nonrecursive system from direct to lattice form by implementing (4.63) and, in cases where $b_0 \neq 1$, modifies the system as in Figure 4.15 and issues a warning that the input signal must be subtracted from the lattice output. The reader may wish to compare the expressions in `nr_dir_to_lat` with (4.63).

A companion function, `nr_lat_to_dir`, converts a nonrecursive lattice to direct form in accordance with (4.62). The output of `nr_lat_to_dir` is a direct weight vector with length one greater than the length of the input lattice weight vector, and with weight b_0 always equal to one.

Another function, `nr_lat_filter`, implements the nonrecursive lattice filter in the form of Figure 4.14. The reader may examine the m-file to see the procedure, which follows essentially from (4.46), in detail.

In this section, we discussed recursive and nonrecursive lattice algorithms and structures. The development was rather long, so we omitted examples, and the reader is encouraged to look at some of the exercises at the end of this chapter on conversion to lattice forms and lattice filtering. One of the best ways to begin with lattices is to convert a simple linear system from direct to lattice form, then process a given signal vector with both forms and see that, although the structures are very different, they produce the same output signal vector.

One final point concerning lattice structures is worth considering: We have seen that in the recursive lattice, the " κ " weights must be less than one for stability. In the nonrecursive lattice, if the κ weights are all less than one, the zeros of $H(z)$ will all be inside the unit circle, resulting in a *minimum-phase* system,¹ which is often desirable. Thus, generally speaking, lattice structures have κ weights less than one; that is, each κ weight is an *attenuator* rather than an amplifier of the signal samples. This property has been, and probably will be, an advantage in DSP systems that operate at the highest possible sampling rates, because, with proper scaling, "hybrid" lattice filters can be made with attenuators that perform analog rather than digital weight multiplication. Hybrid applications continue to exist, because as processor

speeds increase, requirements for greater speeds somehow manage to stay ahead, just out of reach.

4.12 FFT Algorithms

The previous remarks on processor speed requirements bring us to this discussion of *FFT algorithms*, which may also increase the processing speed of a linear system. FFT algorithms are based essentially on (4.8), that is, on the fact that the DFT (or FFT) of the convolution of two vectors is the product of the two individual DFTs (or FFTs).

In any discrete linear system, we have the equivalent relationships in (4.10)–(4.12), that is, $Y(z) = H(z)X(z)$. Also, in connection with (4.25), we have seen that the impulse response, h , is the inverse transform of $H(z)$. If $H(z)$ is nonrecursive, then h is a finite vector equal to b , the weight vector. If $H(z)$ is recursive, h is an infinite vector with elements equal to coefficients of the polynomial resulting from the division of $B(z)$ by $A(z)$. The point here is that, in any case, if $H(z)$ is expressed as a polynomial in z , and if $Y(z) = H(z)X(z)$ in which $x = [x_0 \ x_1 \ \dots \ x_{K-1}]$ is a signal vector, then

$$y_k = \sum_{n=0}^{\infty} h_n x_{k-n}; \quad k = 0, 1, \dots, K-1 \quad (4.64)$$

Furthermore, if we assume as before that x_k is 0 for $k < 0$, then the products in the sum are all zero for $n \geq K$, and

$$y_k = \sum_{n=0}^{K-1} h_n x_{k-n}; \quad k = 0, 1, \dots, K-1 \quad (4.65)$$

Thus, we have the convolution of two finite vectors of equal length (K). (In the nonrecursive system with $h = [b_0 \ b_1 \ \dots \ b_{N-1}]$, we simply append $K - N$ zeros to b in order to obtain h with length K .) We must now ask whether $Y(z)$ can be expressed as a product of transforms in this finite case, as it was for the infinite case in (4.8). To answer, we examine the inverse DFT of the product of two finite DFTs. We use the DFT now instead of the z -transform, because the DFT has discrete components, and the inverse DFT is given in terms of these components. From definition (3.23), the inverse of the DFT product is as follows:

$$\begin{aligned} \text{DFT}^{-1}\{H_m X_m\} &= \frac{1}{K} \sum_{m=0}^{K-1} H_m X_m e^{j2\pi m k / K} \\ &= \frac{1}{K} \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} h_n e^{-j2\pi m n / K} \sum_{i=0}^{K-1} x_i e^{-j2\pi m i / K} e^{j2\pi m k / K} \\ &= \frac{1}{K} \sum_{n=0}^{K-1} \sum_{i=0}^{K-1} h_n x_i \sum_{m=0}^{K-1} e^{j2\pi m (k-i-n)/K}; \quad k = 0, 1, \dots, K-1 \end{aligned} \quad (4.66)$$

$$\begin{array}{lcl}
 \begin{bmatrix} h_0 & h_1 & h_2 & h_3 \\ x_0 & x_3 & x_2 & x_1 \end{bmatrix} & y_0 = & h_0x_0 + h_1x_3 + h_2x_2 + h_3x_1 \\
 \begin{bmatrix} h_0 & h_1 & h_2 & h_3 \\ x_1 & x_0 & x_3 & x_2 \end{bmatrix} & y_1 = & h_0x_1 + h_1x_0 + h_2x_3 + h_3x_2 \\
 \begin{bmatrix} h_0 & h_1 & h_2 & h_3 \\ x_2 & x_1 & x_0 & x_3 \end{bmatrix} & y_2 = & h_0x_2 + h_1x_1 + h_2x_0 + h_3x_3 \\
 \begin{bmatrix} h_0 & h_1 & h_2 & h_3 \\ x_3 & x_2 & x_1 & x_0 \end{bmatrix} & y_3 = & h_0x_3 + h_1x_2 + h_2x_1 + h_3x_0
 \end{array}$$

FIGURE 4.16

Illustration of the periodic convolution of vectors h and x of length $K = 4$, showing the computation of y_0 through y_3 . Compare with Figure 4.1.

In the final expression, the sum on the right is equal to K when $i = k - n \pm vK$, where v is any integer, and is equal to zero otherwise. Therefore, the expression becomes

$$\text{DFT}^{-1}\{H_m X_m\} = \sum_{n=0}^{K-1} h_n x_{k-n \pm vK} \quad k = 0, 1, \dots, K-1; \quad v = 0, 1, \dots, \infty \quad (4.67)$$

Thus, the inverse DFT of a DFT product is essentially the same as in (4.8), except that when the signal vectors h and x are finite, as they are here, h is convolved with the *periodic extension* of x , that is, an infinite vector with the elements of x repeating with period K . [Naturally, in (4.8), we did not notice the periodicity of x , because the period was infinite.] An example of (4.67) similar to Figure 4.1, showing the alignment of h and x with $K = 4$, is shown in Figure 4.16. Alignments are shown in the figure for the computation of y_k with all K values of k .

Comparing Figures 4.16 and 4.1, it is obvious that periodic convolution differs from ordinary convolution, and in this sense, does not represent a linear transfer function. To apply the FFT, we must make periodic convolution look like ordinary convolution. This may be done by appending K zeros to the h and x vectors. Then, again using vectors of length four as an illustration and extending to $K = 8$, the periodic convolution looks like Figure 4.17 instead of Figure 4.16, and the values of y_k in Figure 4.17 now represent the operation of a linear system.

Thus, in general, linear processing may be done using products of FFTs, provided the signal vectors are extended with zeros. In MATLAB, the process takes the following form, in which $x(1:K)$ is the signal vector, and $h(1:K)$ is the system impulse response vector.

$$\begin{aligned}
 y &= \text{IFFT}[\text{FFT}(h, 2 * K) * \text{FFT}(x, 2 * K)] \\
 y &= y(1:K)
 \end{aligned} \quad (4.68)$$